

INKE: Fast Isogeny-Based PKE Using Intermediate Curves

Hyeonhak Kim¹, Seokhie Hong², Suhri Kim³, and Sangjin Lee¹

¹ School of Cybersecurity, Korea University, Seoul 02841, South Korea

² SmartM2M, South Korea

³ School of Mathematics, Statistics and Data Science, Sungshin Women's University, South Korea

Abstract. POKÉ (POint-based Key Exchange), proposed by Basso and Maino at Eurocrypt 2025, is currently the fastest known isogeny-based public-key encryption scheme, combining a SIDH-like protocol with higher-dimensional isogenies. However, the higher-dimensional representation inherently requires discrete logarithm computations, which restricts the use of torsion points to smooth subgroups. As a result, reducing the size of the underlying prime p is challenging, which limits further efficiency gains. In this work, we propose a variant of POKÉ that retains the higher-dimensional representation but avoids discrete logarithm computations. By replacing the point with an intermediate curve as a shared secret, we are able to reduce the size of the prime p and decrease the number of point evaluations, resulting in faster key generation, encryption, and decryption at the cost of a larger public key. We provide optimized C implementations of both POKÉ and our variant. Our results show that the proposed method achieves at least a $1.2\times$ speedup in key generation, $1.03\times$ in encryption, and $2.03\times$ in decryption across all security levels. Using an intermediate curve as the shared secret not only improves efficiency, but also enables a quantum reduction in the Algebraic Isogeny Model (AIM). We show that the security of INKE relies on the quantum hardness of the receiver's and sender's key-recovery problems in the AIM.

Keywords: Post Quantum Cryptography, Isogeny, SIDH, POKÉ, higher dimension

1 Introduction

Isogeny-based cryptography gained attention after the proposal of the Supersingular Isogeny Diffie-Hellman (SIDH) protocol by De Feo and Jao [23]. Although this field dates back to Couveignes' early work in [13], which is currently called the CRS scheme, its use of ordinary elliptic curves posed a critical limitation. Specifically, CRS suffered from both inefficiency and vulnerability to subexponential-time quantum attacks [12]. SIDH, on the other hand, introduced a more practical framework by using supersingular elliptic curves. This not only

improved performance but also offered resistance to previously known quantum attacks at that time [12]. Based on SIDH, the Supersingular Isogeny Key Encapsulation (SIKE) was selected as an alternate candidate in NIST’s post-quantum cryptography standardization project. Until the discovery by Castryck and Decru [8], SIDH was widely believed to be secure.

SIDH-based algorithms use supersingular elliptic curves defined over $\mathbb{F}_{p^2}$, resulting in a non-commutative structure. Therefore, in order for the two parties in the protocol to derive the same shared secret key, each party must evaluate its secret isogeny on the other party’s public base points and transmit the result. This value is referred to as a torsion point. In [8], the authors exploited these exchanged torsion point images and applied Kani’s theorem to recover the private key. Any algorithm that exposes torsion point images of smooth order is susceptible to this attack, leading to the complete breakdown of security in SIDH-based algorithms. Subsequently, Maino et al. [27] and Robert [35] independently demonstrated that SIDH-based algorithms can be solved in polynomial time even when using arbitrary starting curves, making SIDH-based schemes no longer usable.

Since SIDH-based algorithms are among the most efficient in isogeny-based cryptography, various countermeasures have been proposed to mitigate the key recovery attack. In [18], M-SIDH and MD-SIDH were introduced, which are masked variants of SIDH. The key recovery attack exploits the following three aspects of SIDH: 1) publicly known torsion-point information, 2) knowledge of the degree of the isogeny connecting the elliptic curves, and 3) the use of power-smooth isogeny degrees for computational efficiency. By obscuring either the torsion point information or the isogeny degree, the attack becomes much more difficult. Consequently, M-SIDH hides the torsion points by adding random masks, while MD-SIDH conceals the degree of the isogeny used. In [4], binSIDH and terSIDH are proposed that counter the attack. Both binSIDH and terSIDH introduce the concept of artificial orientation and redefine how secret keys are structured. In particular, terSIDH delivers roughly 1.6 bits of security per private key bit and achieves respectable runtime performance, making it a promising candidate among SIDH variants. However, unlike the original SIDH and its variants, where the isogeny degree is independent of the private key, the total isogeny degree of terSIDH depends on the private key. This makes it potentially vulnerable to timing side-channel attacks. Moreover, SIDH variants designed to resist the key recovery attack tend to exhibit slower performance compared to other PQC algorithms or even other isogeny-based schemes, highlighting their limitations in terms of efficiency.

On the other hand, key recovery attacks on SIDH have led to a fundamental shift in the design and computational paradigms of isogeny-based cryptography. The core technique behind the SIDH attack combines Kani’s theorem and Zahrin’s trick, which together enable an isogeny to be embedded into a higher-dimensional isogeny. Traditionally, isogenies were represented by their kernel points, and power-smooth order torsion points were used to efficiently apply Vélú’s formula. However, with the proposal of key recovery attacks, it has been

shown that knowing just the two elliptic curves and the degree of the isogeny connecting them is sufficient to recover the isogeny regardless of the factorization of the underlying isogeny degree. As a result, if one has knowledge of two sets of curves, the associated torsion points, and the isogeny degree, the connecting isogeny can be reconstructed. This implies that isogenies can be expressed in a higher-dimensional form. This newly acquired flexibility in constructing isogenies has introduced greater diversity and improved efficiency, as several existing algorithms have been reinterpreted through higher-dimensional representations such as SQIsign and SCALLOP, yielding more effective and performance results [3,32,16,10,33]. While several SIDH-based schemes—such as Masked SIDH, bin-SIDH, and terSIDH—have been proposed to directly counter the Castryck–Decru attack, recent works have explored combining SIDH-like structures with higher-dimensional isogenies, including IS-CUBE [28], LIT-SiGamal [29], and POKÉ [5].

In [5], Basso and Maino proposed POKÉ, an isogeny-based encryption algorithm that integrates higher-dimensional isogenies into the SIDH structure. The idea lies in enabling two parties—one using a non-smooth, higher-dimensional isogeny with a kernel defined over large extension fields, and the other using a smooth-degree isogeny over $\mathbb{F}_{p^2}$ —to construct a shared commutative diagram and derive a common secret. This construction requires all four elliptic curves involved in the diagram to be public, as one of the isogenies must be identified via both its domain and codomain. The protocol ultimately results in both parties agreeing on a pair of points on a public curve, which is why it is named POKÉ (POint-based Key Exchange). To date, POKÉ currently offers the most efficient performance among isogeny-based encryption algorithms. However, it remains slower than other post-quantum cryptographic schemes, indicating the need for further optimization.

In most public-key cryptosystems, performance is heavily influenced by the size and structure of the underlying finite field. In particular, the efficiency of isogeny-based cryptography depends significantly on the form of the prime, which determines the degrees of the isogenies involved. For example, the prime p in traditional SIDH-based schemes typically takes the form $p = \ell_A^{e_A} \ell_B^{e_B} f \pm 1$, where ℓ_A and ℓ_B are coprime to each other. The common choices for ℓ_A and ℓ_B are 3 and 4, allowing efficient arithmetic. In contrast, countermeasures like M-SIDH and MD-SIDH use different prime structures to resist key recovery attacks. They operate over $\mathbb{F}_{p^2}$ with primes of the form $p = 4\ell_1\ell_2 \cdots \ell_n - 1$, where ℓ_i 's are odd-primes. This choice not only requires computation with high odd-degree isogenies, but also results in inefficient field arithmetic due to the size of the prime. As a result, such schemes suffer significantly slower performance. On the other hand, using a prime of the form $p = 2^a 3^b 5^c f - 1$ is central to the efficiency of POKÉ. This paper aims to further optimize the POKÉ by reducing and refining the structure of such primes for enhanced performance.

1.1 Contributions

To this end, we propose a new variant of POKÉ. Our scheme is called INKE, short for *INtermediate curve-based Key Exchange*. We mainly focus on reducing

the size and altering the structure of the finite field used in POKÉ at the same security level. Since the original construction does not directly translate to primes of different forms, we adapt and modify the scheme accordingly, resulting in a new algorithm tailored to the revised prime structure. The main contributions of this work are as follows:

- **A new variant of POKÉ with smaller finite field.** By adopting the efficient SIDH-like structure of POKÉ and removing the reliance on point evaluations, we propose **INKE**, an optimized variant of POKÉ. In public key cryptography, computational cost is highly influenced by the field size, as smaller fields lead to faster finite field arithmetic. This design choice eliminates the need for point evaluations and allows the use of smaller primes, resulting in significantly faster key generation, encryption, and decryption, at the expense of a larger public key.
- **Security proof of INKE under the Algebraic Isogeny Model (AIM)** For key exchange protocols based on the SIDH structure such as POKÉ, M-SIDH, terSIDH, and binSIDH, security against secret-key recovery does not necessarily imply the security of the key exchange protocol itself, similar to the absence of a generic reduction from the discrete logarithm problem to the computational Diffie–Hellman problem. To address this limitation, it is necessary to consider algebraic adversary models such as the Algebraic Group Model (AGM) and Algebraic Group Action Model (AGAM). Recently, Aardal, Basso, and Riepel [2] formalized a new algebraic model for isogeny-based cryptography, which is called *Algebraic Isogeny Model* (AIM). As the authors have already addressed the general SIDH problem in [2], the security reduction of INKE is also possible in the AIM. Within this model, the structural properties of INKE that uses a curve as the shared secret can be leveraged to construct a security reduction, a technique that is essentially infeasible in POKÉ due to its point-based structure. We show the details of the proof in Section 5.3.
- **Comparative C implementation of POKÉ and INKE.** We provide the first optimized C implementations of POKÉ and INKE. In this regard, we present parameter sets of INKE, targeting NIST security levels 1, 3, and 5. For a fair comparison, we implement both the original POKÉ scheme and INKE in C. Our results show that our scheme achieves over $1.2\times$ and $1.03\times$ improvement in key generation and encryption, respectively, and more than $2.03\times$ improvement in decryption across all security levels. Details are provided in Section 6, and our source code is available at

<https://github.com/gusgkr0117/poke>

We also evaluated the performance of PIKE [7] under the same environment and with the same level of optimization for a fair comparison.

This paper is organized as follows: In Section 2, we present the necessary background, including two-dimensional isogeny representations, the POKÉ algorithm, and its security. Section 3 introduces our proposed algorithm, followed

by a security analysis in Section 5. Implementation results are provided in Section 6, and we conclude in Section 7.

2 Preliminaries

This section provides the background necessary for the paper. First, a two-dimensional representation of an isogeny is presented. Then, POKÉ algorithm is introduced, which is the primary focus of our optimization in this paper.

Throughout this work, p is a prime number and we are interested in supersingular elliptic curves defined over $\mathbb{F}_{p^2}$. An isogeny $\phi : E_1 \rightarrow E_2$ is a non-constant morphism between the elliptic curves E_1 and E_2 , which is a homomorphism between the two elliptic curve groups. The degree of an isogeny is its degree as a rational map, which is the same with the size of the kernel when the isogeny is separable. We refer the reader to [37] for more details.

Let $\phi_1 : E_0 \rightarrow E_1$ and $\phi_2 : E_0 \rightarrow E_2$ be two isogenies with the same domain curve E_0 and coprime degrees. We denote $\phi_{1*}\phi_2 : E_1 \rightarrow E_{12}$ as a pushforward of ϕ_2 under ϕ_1 , i.e. $\ker(\phi_{1*}\phi_2) = \phi_1(\ker \phi_2)$. Then, we have two sets of parallel isogenies $\phi_1 : E_0 \rightarrow E_1$, $\phi_2 : E_0 \rightarrow E_2$, $\phi_{1*}\phi_2 : E_1 \rightarrow E_{12}$ and $\phi_{2*}\phi_1 : E_2 \rightarrow E_{12}$ which form a commutative diagram.

2.1 Two-Dimensional Representation

Theorem 1 (Kani’s Lemma [24]). *Consider the following commutative diagram of isogenies*

$$\begin{array}{ccc} E_0 & \xrightarrow{\phi_1} & E_1 \\ \phi_2 \downarrow & & \downarrow \phi'_2 \\ E_2 & \xrightarrow{\phi'_1} & E_{12} \end{array}$$

where $\deg(\phi_i) = \deg(\phi'_i) = d_i$ and $\gcd(d_1, d_2) = 1$. Then, the isogeny

$$\Phi = \begin{pmatrix} \phi_1 & -\hat{\phi}'_2 \\ \phi_2 & \hat{\phi}'_1 \end{pmatrix} : E_0 \times E_{12} \rightarrow E_1 \times E_2$$

is a $(d_1 + d_2)$ -isogeny whose kernel is given by

$$\text{Ker}\Phi = \{([-d_1]P, \phi'_2 \circ \phi_1(P)) \mid P \in E_0[d_1 + d_2]\}.$$

Kani’s lemma enables the following two-dimensional representation of a certain type of isogenies of degree $q(2^a - q)$ used in FESTA [6] and QFESTA [31]. Let E be an elliptic curve such that $2^a \nmid \#E(\mathbb{F}_{p^2})$. An isogeny $\phi : E \rightarrow E'$ admits an efficient representation

$$(q, E, E', (P_0, Q_0), (\phi(P_0), \phi(Q_0)))$$

where $\deg(\phi) = q(2^a - q)$, $q < 2^a$, $\gcd(q, 2) = 1$ and a basis $\langle P_0, Q_0 \rangle = E[2^a]$. This is called a two-dimensional representation of an isogeny ϕ .

Algorithm 1 describes how we can evaluate a point $R \in E$ through the given two-dimensional representation. As there exists an efficient algorithm to compute the $(2^\ell, 2^\ell)$ -isogeny Φ based on the theta model [14], for the codomain computation and point evaluations of the two-dimensional isogeny in Algorithm 1, we refer to [14, Algorithm 5 and Algorithm 6].

Algorithm 1: Evaluation of a two-dimensional representation

Input: A two-dimensional representation $(q, E, E', (P_0, Q_0), (\phi(P_0), \phi(Q_0)))$
and a point $R \in E(\mathbb{F}_{p^2})$ such that $\gcd(\text{ord}(R), 2) = 1$

Output: $\phi(R)$

Compute Φ with kernel $\langle ([-q]P_0, \phi(P)), ([-q]Q_0, \phi(Q_0)) \rangle$.
Evaluate $(R', \cdot) = \Phi(R, \mathcal{O}_E)$.
Set the order of R as $C \leftarrow \text{ord}(R)$.
Generate a basis U, V of $E'[C]$.
Evaluate $(U', \cdot) = \Phi(\mathcal{O}_{E'}, U)$.
Evaluate $(V', \cdot) = \Phi(\mathcal{O}_{E'}, V)$.
Find x, y such that $R' = [x]U' + [y]V'$. // ▷ By solving ECDLP

1 Compute $\phi(R) = [2^a - q]([x]U + [y]V)$.
2 return $\phi(R)$.

Here, we note that evaluating a point under a two-dimensional representation requires solving a discrete logarithm, which becomes costly when the point has a large non-smooth order. In [5, Remark 6], the authors discuss the necessity of using a smooth order such as 5^c , and the smoothness requirement can be relaxed in another way. The author's alternative approach involves combining two-dimensional isogenies, which requires additional cost. In this work, we propose a different method to address this issue in Section 3.

2.2 POKÉ Protocol

In this section, we briefly introduce the POKÉ protocol. It is based on the SIDH-like structure, which takes place between two parties, Alice and Bob. Alice can compute an isogeny from its domain and codomain curves by using Kani's theorem, while Bob only computes an isogeny represented by a single curve. To avoid the torsion-information-based attack, every point evaluation must involve masking procedures.

Let p be a prime of the form $p = 2^a 3^b 5^c f - 1$ where f is a small cofactor. Let E_0 be the supersingular curve defined over $\mathbb{F}_p$ with j -invariant 1728. Let P_0, Q_0 be a deterministic basis of $E_0[2^a]$, and R_0, S_0 be a deterministic basis of $E_0[3^b]$. Let X_0, Y_0 be a deterministic basis of $E_0[5^c]$. Let KDF denotes a key derivation function.

Key generation The receiver (Alice) samples a random value $q \in [0, 2^a)$ coprime with 30, and computes a random isogeny $\phi : E_0 \rightarrow E_A$ of degree $q(2^a - q)$ by using Algorithm 2. In POKÉ, a rigorous algorithm [5, Algorithm 2] is also proposed to sample $q(2^a - q)$ -isogenies with a distribution identical to the uniform random $q(2^a - q)$ -isogeny distribution. We employ the heuristic version for the efficiency which we call **Gen22Isog** (Algorithm 2). As the distribution of **Gen22Isog** depends on the algorithm **FullRepresentInteger** $_{\mathcal{O}_0}$, we assume that the output of **FullRepresentInteger** $_{\mathcal{O}_0}$ is well distributed in $\mathcal{O}_0$.

Algorithm 2: **Gen22Isog**(p, q) (heuristic)

Input: A degree q , a prime p of the form $p = 2^a 3^b f - 1$.
Output: A representation of pair of isogenies (ϕ_1, ϕ_2) such that $\phi_2 \circ \phi_1 : E_0 \rightarrow E_A$ of degree $q(2^a - q)$.

- 1 Generate an endomorphism $\theta \leftarrow \text{FullRepresentInteger}_{\mathcal{O}_0}(q(2^a - q) \cdot 3^b)$.
// ▷ Using [15, Algorithm 1]
- 2 Compute $P', Q' = \theta(P_0), \theta(Q_0)$.
- 3 Let $K = \ker(\hat{\theta}) \cap E_0[3^b]$.
- 4 Compute $\eta : E_0 \rightarrow E_A$ with kernel K .
- 5 Compute $P_A, Q_A = [1/(3^b)]\eta(P'), [1/(3^b)]\eta(Q')$.
- 6 Compute Φ with kernel $\langle [-q]P_0, P_A \rangle, \langle [-q]Q_0, Q_A \rangle$.
- 7 **return** (ϕ_1, ϕ_2) where $\Phi = \begin{pmatrix} \phi_1 & \hat{\phi}_2 \\ * & * \end{pmatrix}$, $\deg \phi_1 = q$ and $\deg \phi_2 = 2^a - q$.

Algorithm 3: Key Generation

Input: $E_0, P_0, Q_0 \in E_0[2^a], R_0, S_0 \in E_0[3^b], X_0, Y_0 \in E_0[5^c]$.
Output: A secret/public key pair.

- 1 Sample a random $q \in [0, 2^a]$ such that $\gcd(q(2^a - q), 30) = 1$.
- 2 Generate a $q(2^a - q)$ -isogeny $\phi = \phi_2 \circ \phi_1 : E_0 \rightarrow E_A$ from $(\phi_1, \phi_2) \leftarrow \text{Gen22Isog}(p, q)$. // Algorithm 2
- 3 Sample random $\alpha_2, \beta_2 \xleftarrow{\$} \mathbb{Z}_{2^a}^\times$.
- 4 Sample a random $\gamma_3 \xleftarrow{\$} \mathbb{Z}_{3^b}^\times$.
- 5 Sample a random $\delta_5 \xleftarrow{\$} \mathbb{Z}_{5^c}^\times$.
- 6 Compute $P_A, Q_A = [\alpha_2]\phi(P_0), [\beta_2]\phi(Q_0)$.
- 7 Compute $R_A, S_A = [\gamma_3]\phi(S_0), [\gamma_3]\phi(R_0)$.
- 8 Compute $X_A, Y_A = [\delta_5]\phi(X_0), [\delta_5]\phi(Y_0)$.
- 9 **return** $\text{sk} = (q, \alpha_2, \beta_2, \delta_5)$, $\text{pk} = (E_A, P_A, Q_A, R_A, S_A, X_A, Y_A)$.

Encryption The sender (Bob) samples a random integer $r_3 \in \mathbb{Z}_{3^b}$ and computes two isogenies ψ and ψ' of the kernel $\langle R_0 + [r_3]S_0 \rangle$ and $\langle R_A + [r_3]S_A \rangle$, respectively. After generating a shared secret X_{AB}, Y_{AB} from a commutative diagram, it makes the ciphertext $\text{ct} := m \oplus \text{KDF}(X_{AB}, Y_{AB})$ for some fixed key-derivation function KDF. More precisely, encryption is performed as Algorithm 4:

Algorithm 4: Encryption

-
- Input:** A message m , a public key $E_A, P_A, Q_A, R_A, S_A, X_A, Y_A$.
Output: A ciphertext ct .
- 1 Sample a random $r_3 \leftarrow \mathbb{Z}_{3^b}$.
 - 2 Compute the isogeny $\psi : E_0 \rightarrow E_B$ with kernel $\langle R_0 + [r_3]S_0 \rangle$.
 - 3 Compute the isogeny $\psi' : E_A \rightarrow E_{AB}$ with kernel $\langle R_A + [r_3]S_A \rangle$.
 - 4 Sample a random $\omega_2 \leftarrow \mathbb{Z}_{2^a}^\times$.
 - 5 Compute $P_B = [\omega_2]\psi(P_0)$, $Q_B = [1/\omega_2]\psi(Q_0)$.
 - 6 Compute $P_{AB} = [\omega_2]\psi'(P_A)$, $Q_{AB} = [1/\omega_2]\psi'(Q_A)$.
 - 7 Sample a random $D_5 \leftarrow \text{SL}_2(\mathbb{Z}_{5^c})$.
 - 8 Compute $[X_B, Y_B]^T = D_5 \begin{bmatrix} \psi(X_0) \\ \psi(Y_0) \end{bmatrix}$.
 - 9 Compute $[X_{AB}, Y_{AB}]^T = D_5 \begin{bmatrix} \psi'(X_A) \\ \psi'(Y_A) \end{bmatrix}$.
 - 10 Compute $\text{ct}' = \text{KDF}(X_{AB}, Y_{AB}) \oplus m$.
 - 11 **return** $\text{ct} = ((E_B, P_B, Q_B, X_B, Y_B), (E_{AB}, P_{AB}, Q_{AB}), \text{ct}')$.
-

Decryption The receiver computes the points P'_{AB} and Q'_{AB} from the masking value α_2 and β_2 . Then, it recovers the shared secret X_{AB}, Y_{AB} by computing the two-dimensional isogeny Φ' and extracting the message as $m = \text{ct} \oplus \text{KDF}(X_{AB}, Y_{AB})$.

Algorithm 5: Decryption

-
- Input:** $\text{sk} = (q, \alpha_2, \beta_2, \delta_5)$,
 $\text{ct} = ((E_B, P_B, Q_B, X_B, Y_B), (E_{AB}, P_{AB}, Q_{AB}), \text{ct}')$.
Output: A message m' .
- 1 Compute $P'_{AB} = [1/\alpha_2]P_{AB}$, $Q'_{AB} = [1/\beta_2]Q_{AB}$.
 - 2 Compute Φ' with kernel $\langle ([-q]P_B, P'_{AB}), ([-q]Q_B, Q'_{AB}) \rangle$.
 - 3 Evaluate $(X'_B, -) = \Phi'(X_B, \mathcal{O}_{E_{AB}})$.
 - 4 Evaluate $(Y'_B, -) = \Phi'(Y_B, \mathcal{O}_{E_{AB}})$.
 - 5 Generate a basis U, V of $E_{AB}[5^c]$.
 - 6 Evaluate $(U', -) = \Phi'(\mathcal{O}_{E_B}, U)$.
 - 7 Evaluate $(V', -) = \Phi'(\mathcal{O}_{E_B}, V)$.
 - 8 Find x, y such that $X'_B = [x]U' + [y]V'$.
 - 9 Find w, z such that $Y'_B = [w]U' + [z]V'$.
 - 10 Compute $X_{AB} = [\delta_5][2^a - q]([x]U + [y]V)$.
 - 11 Compute $Y_{AB} = [\delta_5][2^a - q]([w]U + [z]V)$.
 - 12 Compute $m' = \text{KDF}(X_{AB}, Y_{AB}) \oplus \text{ct}'$.
 - 13 **return** m' .
-

2.3 POKÉ Security

To ensure the security of the protocol, the following parameters are used :

- $a = \lambda - 2$: Setting a close to the security parameter λ makes brute-force attacks on q computationally hard. An adversary would need to guess q and verify it. For a guessed q , if the adversary succeeds in recovering the random masking value γ_3 on the 3^b -torsion, then it obtains 3^b -torsion information associated with the $(2^a, 2^a)$ -isogeny Φ . To mount a Castryck-Decru-style attack, one must exploit the smoothness of $\deg \Phi$, which in turn requires computing a four-dimensional isogeny. Since computing four-dimensional isogenies is computationally expensive, this cost compensates for the reduction of the search space induced by the constraint $\gcd(q(2^a - q), 30) = 1$ and the additional two bits for using the $(2^a, 2^a)$ -isogeny based on the theta model [14]. This implicitly assumes that a single four-dimensional isogeny computation is more costly than hashing a basis (X_{AB}, Y_{AB}) $40/15 \approx 2.3$ times. This heuristic is motivated by the fact that, one needs to examine up to 15 candidate directions for the first $(2, 2)$ -isogeny step in the Castryck-Decru attack. Although the cost of four-dimensional isogeny computations may continue to decrease with future optimizations, our current parameter setting follows the approach proposed and implemented in POKÉ. We note that the parameter a may therefore be subject to further adjustment once tighter lower bounds on the cost of four-dimensional isogeny computations are established.
- $b = \lceil \log_3(2\lambda) \rceil$: The degree 3^b of the isogenies for encryption is chosen to enable fast encryption. At the same time, it ensures that meet-in-the-middle attacks on ϕ remain exponentially difficult.
- $c = \lceil \log_5(\lambda/3) \rceil$: The parameter 5^c determines the order of the points used in the shared secret. It is chosen to be a power of five to support a fast evaluation of higher-dimensional isogenies and large enough to withstand brute-force attacks. Given a basis U, V of $E_{AB}[5^c]$, recovering the shared secret basis X_{AB}, Y_{AB} requires guessing the change-of-basis matrix. As the pairing of X_{AB}, Y_{AB} is known, one degree of freedom is removed, reducing the attack to guessing three coefficients in $\mathbb{Z}_{5^c}$. This motivates the choice $5^c \approx 2^{\lambda/3}$.

POKÉ has been shown to be secure under the hardness assumption of the custom problem called C-POKE [5].

Problem 1 (C-POKE). Let E_0 be a supersingular elliptic curve defined over $\mathbb{F}_{p^2}$, with known endomorphism ring. Let P_0, Q_0 be a basis of $E_0[2^a]$, R_0, S_0 a basis of $E_0[3^b]$, and X_0, Y_0 a basis of $E_0[5^c]$.

Let $\phi : E_0 \rightarrow E_A$ be an isogeny of degree $q(2^a - q)$, for some unknown value $q \in [0, 2^a]$. Write

$$\begin{aligned} P_A, Q_A &= [\alpha_2] \phi(P_0), & [\beta_2] \phi(Q_0), \\ R_A, S_A &= [\gamma_3] \phi(R_0), & [\gamma_3] \phi(S_0), \\ X_A, Y_A &= [\delta_5] \phi(X_0), & [\delta_5] \phi(Y_0), \end{aligned}$$

where $\alpha_2, \beta_2, \gamma_3, \delta_5$ are uniformly random scalars from

$$\mathbb{Z}_{2^a}^\times \times \mathbb{Z}_{2^a}^\times \times \mathbb{Z}_{3^b}^\times \times \mathbb{Z}_{5^c}^\times.$$

Let $\psi : E_0 \rightarrow E_B$ be an isogeny of degree 3^b , and write $\psi' : E_A \rightarrow E_{AB}$ for its pushforward $\phi_*\psi$. Write

$$\begin{aligned} P_B, Q_B &= [\omega_2] \psi(P_0), \quad [1/\omega_2] \psi(Q_0), \\ P_{AB}, Q_{AB} &= [\omega_2] \psi'(P_A), \quad [1/\omega_2] \psi'(Q_A), \\ X_B, Y_B &= \mathbf{D}_5 [\psi(X_0), \psi(Y_0)]^T, \\ X_{AB}, Y_{AB} &= \mathbf{D}_5 [\psi'(X_A), \psi'(Y_A)]^T, \end{aligned}$$

where ω_2 is a uniformly random scalar from $\mathbb{Z}_{2^a}^\times$, and $\mathbf{D}_5$ is a uniformly random matrix from $\text{SL}_2(\mathbb{Z}_{5^c})$.

Given $(E_0, P_0, Q_0), (R_0, S_0), (X_0, Y_0), (E_A, P_A, Q_A), (R_A, S_A), (X_A, Y_A), (E_B, P_B, Q_B), (X_B, Y_B)$, and (E_{AB}, P_{AB}, Q_{AB}) , compute the points X_{AB}, Y_{AB} .

Theorem 2. *The POKÉ protocol, where the key-derivation function KDF is modeled as a random oracle, is IND-CPA secure in the random oracle model under the assumption that the C-POKE problem is hard.*

Proof. We refer to the proof of [5, Theorem 8].

We can also prove the IND-CPA of the POKÉ protocol in the random oracle model from the hardness of the decisional variant of C-POKE, which is called D-POKE.

3 INKE PKE protocol

In this section, we propose INKE and explain how it enables the use of a smaller prime p in the finite field used in the scheme. Our new scheme resembles the structure of POKÉ, except that we replace the 5^c -torsion points in POKÉ with the intermediate curve of the two-dimensional representation of the secret isogeny ϕ .

In our construction, the receiver (Alice) reveals the intermediate curve along with the evaluated 3^b -torsion basis on this curve. Since the torsion basis is masked and the secret degree q remains hidden, revealing the intermediate curve E_{A_1} , and its associated torsion information does not compromise security. Conceptually, our scheme can be viewed as exposing an additional isogeny and its codomain curve, rather than revealing additional torsion points such as (X_A, Y_A) as done in POKÉ. A detailed description of our protocol is provided below. The overall process is illustrated in Figure 1.

Key generation Given a random value $q \in [0, 2^a]$, we first sample a random isogeny $\phi : E_0 \rightarrow E_A$ of degree $q(2^a - q)$. The authors of POKÉ propose two approaches for sampling such an isogeny: a rigorous approach and a heuristic one. In our case, we adopt the heuristic approach represented in Algorithm 2 for efficiency. The two-dimensional representation of the sampled isogeny $\phi : E_0 \rightarrow E_A$ corresponds to an isogeny commutative diagram involving E_0 and E_A , along

with two intermediate curves E_1 and E_2 such that $\Phi : E_0 \times E_A \rightarrow E_{A_1} \times E_{A_2}$ for the corresponding two-dimensional isogeny Φ .

In our construction, we leverage one of these intermediate curves E_{A_1} or E_{A_2} as a public key of a SIDH-like key exchange protocol (Suppose that E_{A_1} is used), while using E_A as an auxiliary curve to compute the isogeny $\phi_1 : E_0 \rightarrow E_{A_1}$. Consequently, our key generation procedure requires a larger public key compared to that of POKE, as it additionally includes the intermediate curve E_{A_1} and a 3^b -torsion basis (R_{A_1}, S_{A_1}) on this curve. To mitigate the Castryck-Decru attack, the bases (R_{A_1}, S_{A_1}) and (R_A, S_A) are masked independently using random scalars γ'_3 and γ_3 , respectively. Since our scheme employs a j -invariant of an elliptic curve as the shared secret rather than points, our key generation algorithm avoids the use of 5^c -torsion points, which saves some computational overhead.

Algorithm 6: Key Generation

Input: $E_0, P_0, Q_0 \in E_0[2^a]$, $R_0, S_0 \in E_0[3^b]$
Output: A secret/public key pair

- 1 Sample a random $q \in [0, 2^a]$ such that $q \equiv 5 \pmod{6}$.
- 2 Sample a pair of isogenies $(\phi_1, \phi_2) \leftarrow \text{Gen22Isog}(p, q)$ and write $\phi = \phi_2 \circ \phi_1$.
// Algorithm 2
- 3 Sample random $\alpha_2, \beta_2 \leftarrow \mathbb{Z}_{2^a}^\times$.
- 4 Sample a random $\gamma_3, \gamma'_3 \leftarrow \mathbb{Z}_{3^b}^\times$.
- 5 Compute $P_A, Q_A = [\alpha_2]\phi(P_0), [\beta_2]\phi(Q_0)$.
- 6 Compute $R_A, S_A = [\gamma_3]\phi(S_0), [\gamma'_3]\phi(R_0)$.
- 7 Compute E_{A_1} and $R_{A_1}, S_{A_1} = [\gamma'_3]\phi_1(R_0), [\gamma'_3]\phi_1(S_0)$.
- 8 **return** $\text{sk} = (q, \alpha_2, \beta_2)$, $\text{pk} = (E_A, P_A, Q_A, R_A, S_A, E_{A_1}, R_{A_1}, S_{A_1})$.

Algorithm 7: Encryption

Input: A message m , a public key $E_A, P_A, Q_A, R_A, S_A, E_{A_1}, R_{A_1}, S_{A_1}$.
Output: A ciphertext ct .

- 1 Sample a random $r_3 \leftarrow \mathbb{Z}_{3^b}$.
- 2 Compute the isogeny $\psi : E_0 \rightarrow E_B$ with kernel $\langle R_0 + [r_3]S_0 \rangle$.
- 3 Compute the isogeny $\psi' : E_{A_1} \rightarrow E_{A_1B}$ with kernel $\langle R_{A_1} + [r_3]S_{A_1} \rangle$.
- 4 Compute the isogeny $\psi'' : E_A \rightarrow E_{AB}$ with kernel $\langle R_A + [r_3]S_A \rangle$.
- 5 Sample a random $\omega_2 \leftarrow \mathbb{Z}_{2^a}^\times$.
- 6 Compute $P_B = [\omega_2]\psi(P_0)$, $Q_B = [1/\omega_2]\psi(Q_0)$.
- 7 Compute $P_{AB} = [\omega_2]\psi''(P_A)$, $Q_{AB} = [1/\omega_2]\psi''(Q_A)$.
- 8 Compute $\text{ct}' = \text{KDF}(j(E_{A_1B})) \oplus m$.
- 9 **return** $\text{ct} = ((E_B, P_B, Q_B), (E_{AB}, P_{AB}, Q_{AB}), \text{ct}')$.

Encryption In our encryption procedure, we compute three 3^b -isogenies: ψ , ψ' and ψ'' . Compared to POKÉ, our encryption performs an additional isogeny to compute the codomain curve E_{A_1B} , starting from the intermediate curve E_{A_1} , which lies parallel to ψ and ψ'' in the isogeny commutative diagram. Since we use the j -invariant of E_{A_1B} as the shared secret, no additional point evaluation is needed. Although our encryption requires one more isogeny computation than that of POKÉ, the use of a smaller underlying prime significantly improves efficiency, resulting in faster encryption.

Algorithm 8: Decryption

Input: $\text{sk} = (q, \alpha_2, \beta_2)$, $\text{ct} = ((E_B, P_B, Q_B), (E_{AB}, P_{AB}, Q_{AB}), \text{ct}')$
Output: A message m'

- 1 Compute $P'_{AB} = [1/\alpha_2]P_{AB}$, $Q'_{AB} = [1/\beta_2]Q_{AB}$
- 2 Compute Φ' of the kernel $\langle [-q]P_B, P'_{AB} \rangle, \langle [-q]Q_B, Q'_{AB} \rangle$
- 3 **if** $\text{Im}(\Phi')$ *is not splitting* **then abort**
- 4 Compute the codomain curve $E_{A_1B} \times F$ of Φ' .
- 5 Compute $m' = \text{KDF}(j(E_{A_1B})) \oplus \text{ct}'$
- 6 **return** m' .

Decryption In the decryption procedure, given a ciphertext

$$((E_B, P_B, Q_B), (E_{AB}, P_{AB}, Q_{AB}, \text{ct}')),$$

we first recover the points $(P'_{AB}, Q'_{AB}) = ([1/\alpha_2]P_{AB}, [1/\beta_2]Q_{AB})$. To obtain the shared secret curve E_{A_1B} , we compute the two-dimensional isogeny $\Phi' : E_B \times E_{AB} \rightarrow E_{A_1B} \times F$ with the kernel

$$\langle [-q]P_B, P'_{AB} \rangle, \langle [-q]Q_B, Q'_{AB} \rangle,$$

and recover the message m' from ct' with $j(E_{A_1B})$. As our scheme does not require point evaluations or discrete logarithm computations, the decryption process is considerably more efficient than that of POKÉ.

The entire keygen, encryption, and decryption procedures are depicted in Figure 1. Our diagram is very similar to the diagram of POKÉ [5, Figure 2], except that we use the intermediate curves E_{A_1} and E_{A_1B} .

4 How to identify E_{A_1B} ?

During the decryption process, we compute a $(2^a, 2^a)$ -isogeny $\Phi' : E_A \times E_{AB} \rightarrow E_{A_1B} \times F_B$ and we must identify the right curve E_{A_1B} which corresponds to the curve E_{A_1} , i.e. $j(\text{Im}(\phi_{1*}\psi)) = j(E_{A_1B})$ in Fig. 1. A straightforward approach is to evaluate two points and compare the pairings. For example, we

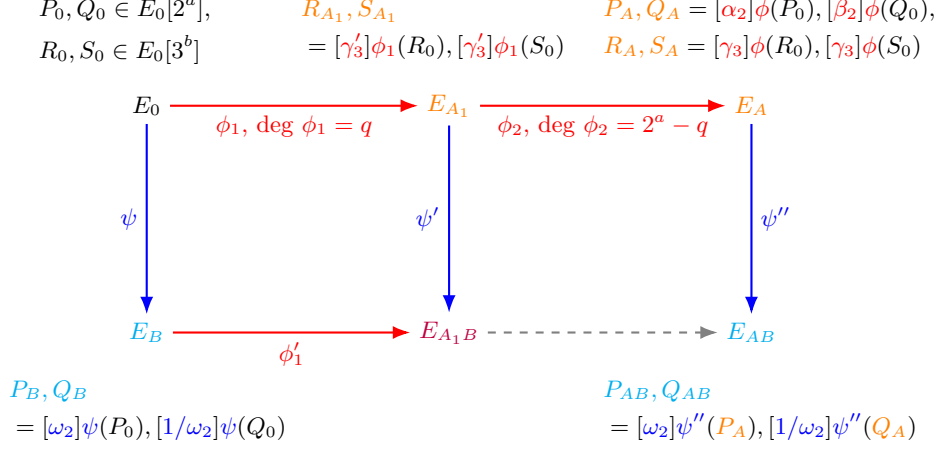


Fig. 1: Our INKE protocol. Public parameters are in black; elements in **red** are part of the secret key or computed during decryption; **orange** parts are public keys; **blue** are ephemeral secrets; **light blue** are ciphertext parts; **purple** shows the shared secret.

choose a random N -torsion basis $U_1, V_1 \in E_B$ and evaluate these two points by $(U_2, U_3) = \Phi'(U_1, \mathcal{O})$ and $(V_2, V_3) = \Phi'(V_1, \mathcal{O})$. Then, we can distinguish the two curves E_{A_1B} and F_B by solving the discrete logarithm $e(U_2, V_2)^x = e(U_1, V_1)$ and $e(U_3, V_3)^y = e(U_1, V_1)$ and obtaining $x \equiv \deg \phi_1 \pmod N$ and $y \equiv \deg \phi_2 \pmod N$. We choose the smallest N such that $\deg \phi_1 \pmod N$ and $\deg \phi_2 \pmod N$ are different. This method requires pairing and discrete logarithm computations, which are quite expensive.

Fortunately, the most efficient $(2^a, 2^a)$ -isogeny computation based on the theta model [14] allows us to naturally distinguish the two curves without additional pairing computations. In the theta-model-based approach, isogeny computation relies on extra 4-torsion points to construct a theta structure and to work with the associated theta (Heisenberg) group. The automorphisms preserving a fixed theta structure induce symplectic automorphisms on the associated theta group [14]. The action of the symplectic automorphism on theta coordinates is constrained and does not allow switching of the two curves in an elliptic curve product. As a consequence, an intrinsic ordering is imposed on an elliptic curve product in the theta model. When traversing a two-dimensional isogeny diamond and returning to the starting surface, the induced transformation is a symplectic automorphism, which preserves this ordering of the elliptic curve product. From this perspective, the theta model can be viewed as providing a clean alternative to explicit pairing computations with extra torsion points, encoding the relevant information directly through the structure of the theta group.

5 Security of Our Scheme

In this section, we analyze the security of our scheme, which relies on the hardness of the computational problem that asks to find the shared secret given all public details. In our setting, the underlying hardness assumption is formalized as Definition 1, C-INKE, which closely resembles the C-POKE problem introduced in [5], except that the 5^c -torsion information is omitted and additional 3^b -torsion information, along with a new intermediate curve E_{A_1} , is given.

Definition 1 (C-INKE). For an algorithm $\mathcal{A}$, we define the advantage of finding the codomain curve of the isogeny $\phi_{1*}\psi$, given the output of Experiment 1 as follows:

$$\text{Adv}^{\text{C-INKE}} := \Pr[j(\text{Im}(\phi_{1*}\psi)) = j' : j' \leftarrow \mathcal{A}(x) \wedge x \leftarrow \text{C-INKE}(1^\lambda)]$$

Experiment 1: C-INKE Problem

C-INKE(1^λ)

1. Let $q \xleftarrow{\$} [0, 2^a)$ such that $q \equiv 5 \pmod{6}$.
2. Sample a pair of isogenies $(\phi_1, \phi_2) \leftarrow \text{Gen2Isog}(p, q)$ and let $\phi = \phi_2 \circ \phi_1 : E_0 \rightarrow E_A$.
3. Choose uniformly random scalars $(\alpha_2, \beta_2, \gamma_3, \gamma'_3) \in \mathbb{Z}_{2^a}^\times \times \mathbb{Z}_{2^a}^\times \times \mathbb{Z}_{3^b}^\times \times \mathbb{Z}_{3^b}^\times$.
4. Compute masked bases $(P_A, Q_A) = ([\alpha_2]\phi(P_0), [\beta_2]\phi(Q_0))$, $(R_A, S_A) = ([\gamma_3]\phi(R_0), [\gamma_3]\phi(S_0))$ and $(R_{A_1}, S_{A_1}) = ([\gamma'_3]\phi_1(R_0), [\gamma'_3]\phi_1(S_0))$.
5. Choose a uniformly random cyclic isogeny $\psi : E_0 \rightarrow E_B$ of degree 3^b .
6. Let $\psi'' = \phi_*\psi : E_A \rightarrow E_{AB}$.
7. Choose a uniformly random scalar $\omega \in [0, 2^a)$.
8. Compute masked bases $(P_B, Q_B) = ([\omega_2]\psi(P_0), [1/\omega_2]\psi(Q_0))$ and $(P_{AB}, Q_{AB}) = ([\omega_2]\psi''(P_A), [1/\omega_2]\psi''(Q_A))$.
9. Return $(E_0, (P_0, Q_0), (R_0, S_0), E_{A_1}, (R_{A_1}, S_{A_1}), E_A, (P_A, Q_A), (R_A, S_A), E_B, (P_B, Q_B), E_{AB}, (P_{AB}, Q_{AB}))$.

Theorem 3. Suppose that the key-derivation function of INKE is modeled as a random oracle. For any PPT adversary $\mathcal{A}$ for the IND-CPA of INKE in a random oracle model, there exists a PPT algorithm $\mathcal{B}$ such that

$$\text{Adv}_{\text{INKE}}^{\text{IND-CPA}}(\mathcal{A}) \leq q_g \cdot \text{Adv}^{\text{C-INKE}}(\mathcal{B}),$$

where q_g is the number of random oracle queries.

Proof. Our proof follows a structure similar to that of [25, Theorem 1], which is also adopted in the security analysis of POKÉ. Suppose there exists a polynomial-time algorithm $\mathcal{A}$ that breaks the IND-CPA security of our scheme. We show

how to construct a polynomial-time algorithm $\mathcal{B}$ that solves the C-INKE problem using $\mathcal{A}$.

We assume that the KDF of our scheme is modeled as a random oracle. After $\mathcal{A}$ outputs two messages m_1 and m_2 as challenge candidates, $\mathcal{B}$ responds with the ciphertext $((E_B, P_B, Q_B), (E_{AB}, P_{AB}, Q_{AB}), m_b \oplus r)$ where r is chosen uniformly at random from the message space and $b \in \{0, 1\}$ is selected randomly. If $\mathcal{A}$ does not query the random oracle at the value $j(E_{A_1B})$, then the probability that $\mathcal{A}$ distinguishes between $m_b \oplus r$ and m_b is negligible.

If $\mathcal{A}$ queries the random oracle at the value $j(E_{A_1B})$, then $\mathcal{B}$ can use this query to solve the C-INKE problem. If $\mathcal{A}$ makes at most q_g random-oracle queries, then the probability that $\mathcal{B}$ correctly guesses the relevant query is at least $1/q_g$. Therefore, $\mathcal{B}$ solves the C-INKE problem with probability $1/q_g$, completing the reduction. $\square$

It is also possible to construct an IND-CCA secure KEM in the Quantum Random Oracle Model (QROM) by applying the Fujisaki-Okamoto (FO) transformation [19], combined with the modular proof framework of [21, Theorem 3.2 and Theorem 3.3]. We note that the proof of Theorem 3 is not tight. It loses the factor of q_g , the number of random oracle queries. As mentioned in POKÉ, it is also possible to obtain a tighter proof of IND-CPA security in the random oracle model by relying on the hardness of the decisional version problem.

5.1 Analysis of Receiver's Security

The following problem is our modified version of the receiver's security of POKÉ [5, Problem 11] for security against key-recovery attacks.

Definition 2 (Receiver's Security). *For an algorithm $\mathcal{A}$, we define the advantage of finding ϕ , given the output of $\text{Recv}(1^\lambda)$ (Experiment 2) as $\text{Adv}^{\text{Recv}}(\mathcal{A})$.*

Experiment 2: Receiver's Security

$\text{Recv}(1^\lambda)$

1. Choose a random integer $q \in [0, 2^a)$ such that $q \equiv 5 \pmod{6}$.
2. Sample a pair of isogenies $(\phi_1, \phi_2) \leftarrow \text{Gen22Isog}(p, q)$ and let $\phi = \phi_2 \circ \phi_1 : E_0 \rightarrow E_A$.
3. Choose uniformly random scalars $(\alpha_2, \beta_2, \gamma_3, \gamma'_3) \in \mathbb{Z}_{2^a}^\times \times \mathbb{Z}_{2^a}^\times \times \mathbb{Z}_{3^b}^\times \times \mathbb{Z}_{3^b}^\times$.
4. Compute masked bases

$$\begin{aligned} (P_A, Q_A) &= ([\alpha_2]\phi(P_0), [\beta_2]\phi(Q_0)) \\ (R_A, S_A) &= ([\gamma_3]\phi(R_0), [\gamma_3]\phi(S_0)) \\ (R_{A_1}, S_{A_1}) &= ([\gamma'_3]\phi_1(R_0), [\gamma'_3]\phi_1(S_0)). \end{aligned}$$

5. Return $E_0, (P_0, Q_0), (R_0, S_0), E_{A_1}, (R_{A_1}, S_{A_1}), E_A, (P_A, Q_A), (R_A, S_A)$.

The receiver's security of INKE can be viewed as a variant of that of POKÉ, with additional information of an intermediate curve E_{A_1} , and its basis (R_1, S_1) . Consequently, all aspects of the receiver's security analysis for POKÉ in [5, Section 6.1] apply directly to INKE as well. For example, in POKÉ it is shown that without diagonal scaling, masking $\phi(P_0)$ and $\phi(Q_0)$ leaks information about the isogeny, while with diagonal scaling the isogeny is perfectly concealed. This observation equally applies in the INKE setting. In this section, we focus only on the additional issues that may arise in INKE due to the exposure of the intermediate curve.

Exploiting the intermediate curve E_{A_1} One may consider a meet-in-the-middle (MITM) attack to find the isogeny between E_0 and the intermediate curve E_1 (or from E_1 to E_2). Since the degree of ϕ_1 is neither fixed nor smooth, an attacker would need to know the codomain surface $E_1 \times F$ of the two-dimensional isogeny $\Phi : E_0 \times E_2 \rightarrow E_1 \times F$. If the curve F were exposed, the attacker could perform an MITM attack on the two-dimensional $(2, 2)$ -isogeny graph, as the degree of Φ is fixed and known as $(2^a, 2^a)$. However, there is no known polynomial-time algorithm to recover the hidden curve F from the given three curves E_0, E_1 , and E_2 arising from an isogeny diamond, even if one of the isogenies ϕ_1 and ϕ_2 is given. This problem is closely related to the Decisional Supersingular Product (DSSP) problem introduced in [23].

Exploiting the two-dimensional torsion information Since the degree of ϕ_1 is not fixed, any attack using the torsion information associated with ϕ_1 is infeasible. Therefore, it is natural to consider exploiting torsion information in the higher-dimensional setting, since the two-dimensional isogeny Φ has a fixed degree of $(2^a, 2^a)$.

From the perspective of the two-dimensional isogeny $\Phi : E_0 \times E_2 \rightarrow E_1 \times F$, the setting of Problem 2 can be interpreted as a partial exposure of the masked torsion information $[\gamma'_3]\Phi((R_0, 0_{E_2}))$ and $[\gamma'_3]\Phi((S_0, 0_{E_2}))$, with the degree of Φ known to be 2^a .

Given the abelian surface $A = E_0 \times E_2$ equipped with the product polarization, we have the pairing [30, Section 23, p.228]

$$\begin{aligned} e_A : A[3^b] \times A[3^b] &\rightarrow \mu_{3^b} \\ e_A((P, Q), (R, S)) &= e_{E_0}(P, R) \cdot e_{E_2}(Q, S) \end{aligned}$$

where $\mu_n = \{z \in \overline{\mathbb{F}_p}^\times : z^n = 1\}$. Given (R_0, S_0) and (R_1, S_1) , one may attempt to recover the masking factor γ'_3 by computing the two-dimensional pairing and exploiting the following relations:

$$\begin{aligned} e_A((R_0, 0_{E_2}), (S_0, 0_{E_2})) &= e_{E_0}(R_0, S_0) \cdot e_{E_2}(0_{E_2}, 0_{E_2}) = e_{E_0}(R_0, S_0) \\ e_A((R_0, 0_{E_2}), (S_0, 0_{E_2}))^{\deg \Phi} &= e_A(\Phi(R_0, 0_{E_2}), \Phi(S_0, 0_{E_2})) \\ e_A(\Phi(R_0, 0_{E_2}), \Phi(S_0, 0_{E_2})) &= e_{E_1}(R_1, S_1)^{\gamma'^{-2}_3} \cdot e_{E_0}(R_0, S_0)^{\deg \phi_2}. \end{aligned}$$

By solving the discrete logarithm $e_{E_0}(R_0, S_0)^x = e_{E_1}(R_1, S_1)$ in the smooth-order of 3^b , one can obtain

$$x = \gamma_3'^2 \cdot (2^a - \deg \phi_2) \pmod{3^b}.$$

However, since $\deg \phi_2$ is randomly chosen, γ_3' is hidden, and the distribution of x is uniformly random and independent of γ_3' . As noted in [5, Section 6.1], by solving the discrete logarithm $e_{E_0}(R_0, S_0)^y = e_{E_2}(R_2, S_2)$, we also obtain

$$y = \gamma_3^2 \cdot \deg \phi_2 \cdot (2^a - \deg \phi_2) \pmod{3^b}.$$

Thus, we have $y/x \equiv \frac{\gamma_3^2}{\gamma_3'^2} \cdot \deg \phi_2 \pmod{3^b}$. This reveals the quadratic residuosity of q and $2^a - q$ in modulus 3^b . However, in the settings of both POKÉ and INKE, the condition $\gcd(q(2^a - q), 3) = 1$ implies that $q \equiv 2 \pmod{3}$ when a is even. Under this constraint, both q and $2^a - q$ are necessarily quadratic non-residues modulo 3^b . Consequently, the pairing does not reveal any additional information about q , since the quadratic residuosity of both values is already fixed. In contrast, POKÉ imposes an additional constraint $\gcd(q(2^a - q), 5) = 1$, which reduces the search space for q by a factor of $3/5$. As a result, INKE leaks less information than POKÉ, with the difference corresponding to approximately 0.74-bit of security.

5.2 Analysis of Sender's Security

The following problem is our modified version of [5, Problem 12] for the security against key-recovery attacks to the sender's isogenies.

Definition 3. For an algorithm $\mathcal{A}$, we define the advantage of finding ψ , given the output of $\text{C-INKE}(1^\lambda)$ (Experiment 1) as $\text{Adv}^{\text{Send}}(\mathcal{A})$.

Our sender's security is identical to that of POKÉ except that the 5^c -torsion information is not revealed, which means that the sender problem in POKÉ reduces to the sender problem of INKE. In POKÉ, the authors formalized the relationship between their sender problem and the CIST^2 problem [31, Problem 3] by showing that both have a reduction from an instance of the CIST problem (Definition 4). In the same spirit, we show a reduction from the same problem to our sender problem. Here, we formalize Problem 13 from the POKÉ paper [5] and refer to it as the *stronger version of the sender's security* (**Str-Send**), as it captures the sender's security even in the stronger setting where the receiver's secret key is given.

Experiment 3: (Strong) Sender's SecurityStr-Send(1^λ)

1. Choose a random integer $q \in [0, 2^a)$ such that $q \equiv 5 \pmod{6}$.
2. Sample a pair of isogenies $(\phi_1, \phi_2) \leftarrow \text{Gen22Isog}(p, q)$ and let $\phi = \phi_2 \circ \phi_1 : E_0 \rightarrow E_A$.
3. Choose a uniformly random cyclic isogeny $\psi : E_0 \rightarrow E_B$ of degree 3^b .
4. Compute $\psi'' = \phi_* \psi : E_A \rightarrow E_{AB}$.
5. Choose a uniformly random scalar $\omega_2 \in \mathbb{Z}_{2^a}^\times$.
6. Compute a masked basis $(P_B, Q_B) = ([\omega_2]\psi(P_0), [1/\omega_2]\psi(Q_0))$.
7. Return $E_0, (P_0, Q_0), \phi, E_B, (P_B, Q_B), \phi' = \psi_* \phi$.

Definition 4 (Problem 13 in [5]). For an algorithm $\mathcal{A}$, we define the advantage of finding ψ , given the output of **Str-Send** (Experiment 3) as $\text{Adv}^{\text{Str-Send}}(\mathcal{A})$.

Remark 1. From any efficient representation of isogenies ϕ and ϕ' output by **Str-Send**(1^λ), one can recover their corresponding two-dimensional representations. Here, by an *efficient representation* of an isogeny, we mean that point evaluation can be performed in polynomial time. Since $\deg \phi = q(2^a - q) < p + 1$, the degree of ϕ can be determined by evaluating ϕ on a basis $P, Q \in E_0(\mathbb{F}_{p^2})$. Given $\deg \phi$, the parameter q can be recovered by solving the quadratic equation $q(2^a - q) = \deg \phi$. This uniquely determines q , and consequently allows one to reconstruct the associated two-dimensional representation.

Theorem 4 (Theorem 14 in [5]). For any PPT adversary $\mathcal{A}$ for the **Send** problem, there exists a PPT algorithm $\mathcal{B}$ such that

$$\text{Adv}^{\text{Send}}(\mathcal{A}) \leq \text{Adv}^{\text{Str-Send}}(\mathcal{B})$$

Proof. By using Remark 1, the adversary $\mathcal{B}$ can recover the two-dimensional representation (ϕ_1, ϕ_2) of ϕ . The adversary $\mathcal{B}$ samples uniformly random scalars $\alpha_2, \beta_2 \in \mathbb{Z}_{2^a}^\times$, $\gamma, \gamma' \in \mathbb{Z}_{3^b}^\times$. Then, $\mathcal{B}$ chooses a random 3^b -torsion basis $R_0, S_0 \in E_0[3^b]$ and computes

$$\begin{aligned} (R_{A_1}, S_{A_1}) &= ([\gamma']\phi_1(R_0), [\gamma']\phi_1(S_0)) \\ (R_A, S_A) &= ([\gamma]\phi(R_0), [\gamma]\phi(S_0)) \\ (P_A, Q_A) &= ([\alpha_2]\phi(P_0), [\beta_2]\phi(Q_0)) \\ (P_{AB}, Q_{AB}) &= ([\alpha_2]\phi'(P_B), [\beta_2]\phi'(Q_B)). \end{aligned}$$

$\mathcal{B}$ runs $\mathcal{A}$ with an input

$$(E_0, (P_0, Q_0), (R_0, S_0)), (E_{A_1}, (R_{A_1}, S_{A_1})), (E_A, (P_A, Q_A), (R_A, S_A)), \\ (E_B, (P_B, Q_B)), (E_{AB}, (P_{AB}, Q_{AB})).$$

The adversary $\mathcal{B}$ then returns the same output produced by $\mathcal{A}$. $\square$

As noted in the POKÉ paper [5], the **Str-Send** problem is very similar to the **CIST** problem. The given isogeny and its pushforward are represented by a kernel point of ϕ and its evaluated point, up to scalar. The **Str-Send** problem can be thought of as an instance of the **CIST** problem where the order of torsion points is $q(2^a - q)$.

5.3 Security Proof of INKE in the Algebraic Isogeny Model

In INKE, assuming both the receiver's security (Definition 2) and the sender's security (Definition 3), we establish that the **C-INKE** problem is secure against algebraic adversaries by using a quantum reduction. Building on the recent security framework of Aardal, Basso and Riepel [2], we apply their general DLOG-to-CDH reduction proof [2, Theorem 6.7] to our setting. The Algebraic Isogeny Model (AIM) is a generalization of the Algebraic Group Action Model (AGAM) allowing an adversary to compute elliptic curves via arbitrary isogenies rather than group elements. This framework enables security proofs of SQISign [1] and general SIDH-like protocols against algebraic adversaries in both quantum and classical settings. For further details on AIM, we refer the reader to [2].

However, in the case of that the starting curve E_0 is fixed, the DLOG-to-CDH reduction of Aardal, Basso, and Riepel does not yield a fully satisfactory reduction solely based on sender and receiver's key recovery assumptions. It requires the hardness of computing a non-scalar endomorphism on a party's image curve.

Recent work by Mingjie Chen and Christophe Petit [11] shows that, given a full-rank suborder of the endomorphism ring of a supersingular curve, there exists a quantum polynomial time algorithm that recovers the entire endomorphism ring. Moreover, they prove that, given the endomorphism ring of a domain curve together with an isogeny path, there is a quantum polynomial time algorithm that computes the endomorphism ring of the codomain curve (generalization of the result in [9]).

By combining these as algorithmic tools, we show that the **C-INKE** problem admits a quantum reduction that relies solely on the sender's and receiver's security assumptions.

Prior to the reduction proof, we first clarify what constitutes an efficient representation of an isogeny. In particular, we adopt the notion of a weak isogeny representation introduced in [11].

Definition 5 (Efficient isogeny representation [2]). *An efficient isogeny representation for the isogeny $\varphi : E \rightarrow E'$, is a string s_φ of size $O(\text{poly}(\log(p + \deg \varphi)))$ such that there exists a pair of polynomial time algorithms:*

- $\mathcal{E}.\text{valid}(s_\varphi)$: *On input a string s_φ , it outputs $\perp$ or a triple (E, E', d) . In the latter case, E and E' are elliptic curves defined over $\mathbb{F}_p$ and s_φ represents an isogeny $\varphi : E \rightarrow E'$ of degree d .*
- $\mathcal{E}.\text{eval}(s_\varphi, P)$: *On input a string s_φ and a point $P \in E(\mathbb{F}_{q^k})$, it outputs the image point $\varphi(P) \in E'(\mathbb{F}_{q^k})$ if $\mathcal{E}.\text{valid}(E) = (E, E', d)$, otherwise the output is undefined.*

In the case that $\mathcal{E}.\text{valid}(s_\varphi) \neq \perp$, we say that s_φ is an efficient representation of φ (with respect to $\mathcal{E}$).

Definition 6. In an algebraic model for isogenies, an algorithm $\mathcal{A}$ said to be algebraic as follows. Let $\mathcal{A}$ gets as input the curves $E_1, \dots, E_n$. If $\mathcal{A}$ outputs a curve E , it also outputs an efficient representation $(s_{\varphi_{\text{expl}}}, \mathcal{E})$ of an isogeny $\varphi_{\text{expl}} : E_i \rightarrow E$ for some $1 \leq i \leq n$ or $\varphi_{\text{expl}} : E_{\text{End}} \rightarrow E$ where E_{End} is a curve with known endomorphism ring. We call the isogeny φ_{expl} the explanation for E .

For the reduction proof, we also introduce the DIVIDE algorithm to extract the secret information from the adversary's isogeny explanation φ_{expl} . The DIVIDE algorithm takes as input an isogeny φ and an integer N , then outputs φ/N when $N \mid \deg \varphi$.

Theorem 5 (Theorem 3 in [20]). There is an algorithm DIVIDE which takes as input

- Two supersingular elliptic curves E_1, E_2 defined over $\mathbb{F}_{p^2}$,
- an isogeny $\varphi : E_1 \rightarrow E_2$ in efficient representation,
- an integer $n < \det \varphi$,

and returns an efficient representation of φ/N if this quotient is an isogeny, and $\perp$ otherwise. The algorithm runs in time polynomial in $k \log p$ and $\log \deg \varphi$.

Definition 7 (ℓ -reduced isogeny). An isogeny φ is said to be ℓ -reduced if φ/ℓ is not an isogeny.

Lemma 5.1. Let $\psi : E_{A_1} \rightarrow E_{A_1 B}$ be a cyclic isogeny of degree 3^b . Let $\varphi : E_{A_1} \rightarrow E_{A_1 B}$ be an 3 -reduced isogeny of arbitrary degree. Let $\sigma = \widehat{\varphi} \circ \psi \in \text{End}(E_{A_1})$. If $\ker \sigma \cap E_{A_1}[3^b] = E_{A_1}[3^b]$, then $\ker \widehat{\varphi} \cap E_{A_1}[3^b] = \ker \widehat{\psi}$. That means that φ is extractable by using the ExtractSend algorithm in the context of the C-INKE problem.

Proof. Let $\ker \sigma \cap E_{A_1}[3^b] = E_{A_1}[3^b]$. Then, we have

$$\ker \widehat{\varphi} \supseteq \psi(E_{A_1}[3^b]) = \ker \widehat{\psi}.$$

Since the isogeny φ is 3 -reduced, $\ker \widehat{\varphi} \cap E_{A_1 B}[3^k] \neq E_{A_1 B}[3^k]$ for all $k \in \mathbb{N}$, which means that $\widehat{\varphi} \cap E_{A_1 B}[3^b]$ is cyclic and equals to $\ker \widehat{\psi}$. $\square$

Lemma 5.2. Let $x \leftarrow \text{Recv}(1^\lambda)$. Let $(\psi_1, \psi'_1, \psi''_1, y_1)$ and $(\psi_2, \psi'_2, \psi''_2, y_2)$ be outputs from $\text{ExtendRecv}(x)$ by running it twice. Let $\varphi_i : E_{A_1} \rightarrow E_i$ be two 3 -reduced isogenies of arbitrary degree ($i = 1, 2$). Let $\sigma_i = \widehat{\varphi}_i \circ \psi'_i \in \text{End}(E_{A_1})$. If both $\text{ExtractSend}(\varphi_i, y_i)$ abort, the probability that the subring of $\text{End}(E_{A_1})$ generated by $\{1, \sigma_1, \sigma_2\}$ has rank 4 is at least $1/2$.

Algorithm 9: Algorithm ExtractSend

Input: An efficient representation $(s_\varphi, \mathcal{E})$ of an isogeny $\varphi : E \rightarrow E_{A_1B}$ and an associated instance x of the C-INKE problem.

Output: The sender's secret isogeny $\psi : E_0 \rightarrow E_B$.

- 1 Parse x and obtain

$$E_0, (P_0, Q_0), (R_0, S_0), E_{A_1}, (R_{A_1}, S_{A_1}), E_A, (R_A, S_A), (P_A, Q_A),$$

$$E_B, (P_B, Q_B), E_{AB}, (P_{AB}, Q_{AB}).$$
- 2 Set $z \leftarrow \varphi$.
- 3 **repeat**
- 4 Set $\varphi \leftarrow z$.
- 5 Compute $z \leftarrow \text{DIVIDE}(\varphi, 3)$. // Theorem 5
- 6 **until** $z = \perp$
 // Repeat until z is 3-reduced
- 7 Compute the subgroup $G = \ker \widehat{\varphi} \cap E_{A_1B}[3^b]$ by running $\mathcal{E}.\text{eval}(s_\varphi, E[3^b])$.
- 8 **if** G is not a cyclic subgroup of order 3^b **then aborts**.
- 9 Compute a kernel representation of the isogeny $\widehat{\psi}' : E_{A_1B} \rightarrow E' = E_{A_1B}/G$.
- 10 Compute $\delta \in \mathbb{Z}_{3^b}^\times$ such that $\ker \psi' = \langle R_{A_1} + [\delta]S_{A_1} \rangle$ and compute $\psi'' : E_A \rightarrow E''$ such that $\ker \psi'' = \langle R_A + [\delta]S_A \rangle$.
- 11 **if** $j(E') \neq j(E_{A_1})$ or $j(E'') \neq j(E_{AB})$ **then aborts**.
- 12 **if** $\langle \psi'(P_0) \rangle \neq \langle P_B \rangle$ or $\langle \psi'(Q_0) \rangle \neq \langle Q_B \rangle$ or $\langle \psi''(P_A) \rangle \neq \langle P_{AB} \rangle$ or $\langle \psi''(Q_B) \rangle \neq \langle Q_{AB} \rangle$ **then aborts**.
- 13 Compute $\psi : E_0 \rightarrow E_B$ such that $\ker \psi = \langle R_0 + [\delta]S_0 \rangle$.
- 14 **return** $\psi : E_0 \rightarrow E_B$.

Proof. By Lemma 5.1, if $\text{ExtractSend}(\varphi_i, y_i)$ aborts, i.e., is not extractable, then

$$E_{A_1}[3^b] \supsetneq \ker \sigma_i \cap E_{A_1}[3^b] \supseteq \ker \psi'_i.$$

Since $E_{A_1}[3^b] \cong \mathbb{Z}_{3^b} \times \mathbb{Z}_{3^b}$, the endomorphism σ_i can be represented by a matrix

$$M_i \in M_{2 \times 2}(\mathbb{Z}_{3^b}) \setminus \{\mathcal{O}\}.$$

As $\ker M_i$ contains a cyclic group of order 3^b , it follows that $\det(M_i) \equiv 0 \pmod{3}$. Suppose that the subring generated by $\{1, \sigma_1, \sigma_2\}$ has rank 2. Reducing modulo 3, its rank is at most 2 over $\mathbb{Z}_3$. Hence, in the induced representation on the 3-torsion subgroup, there exist $a, b, c \in \mathbb{Z}_3$, not all zero, such that

$$aI + b\tilde{M}_1 + c\tilde{M}_2 = \mathcal{O} \in M_{2 \times 2}(\mathbb{Z}_3),$$

where $\tilde{M}_i = M_i \pmod{3}$. Taking determinants and using $\det(\tilde{M}_i) = 0$, we obtain

$$c^2 \det(\tilde{M}_2) = \det(aI + b\tilde{M}_1) = a^2 + ab \operatorname{tr}(\tilde{M}_1) + b^2 \det(\tilde{M}_1) = a(a + b \operatorname{tr}(\tilde{M}_1)) = 0.$$

Thus, either $a = 0$ or $a = -b \operatorname{tr}(\tilde{M}_1)$.

If $a = 0$, then $c\tilde{M}_2 = b\tilde{M}_1$, and hence $\ker(\tilde{M}_2) = \ker(\tilde{M}_1)$.

Algorithm 10: Algorithm ExtendRecv**Input:** A Recv instance x .**Output:** Kernel representations of the three cyclic 3^b -isogenies $\psi : E_0 \rightarrow E_B$, $\psi' : E_{A_1} \rightarrow E_{A_1B}$, $\psi'' : E_A \rightarrow E_{AB}$, and an associated C-INKE instance x .1 Parse x and obtain

$$E_0, (P_0, Q_0), (R_0, S_0), E_{A_1}, (R_{A_1}, S_{A_1}), E_A, (R_A, S_A), (P_A, Q_A).$$

2 Choose a random scalar $\delta \in \mathbb{Z}_{3^b}^\times$.3 Compute 3^b -isogenies $\psi : E_0 \rightarrow E_B$, $\psi' : E_{A_1} \rightarrow E_A$ and $\psi'' : E_A \rightarrow E_{AB}$ such that $\ker \psi = \langle R_0 + [\delta]S_0 \rangle$, $\ker \psi' = \langle R_{A_1} + [\delta]S_{A_1} \rangle$ and $\ker \psi'' = \langle R_A + [\delta]S_A \rangle$.4 Choose a random scalar $\omega_2 \in \mathbb{Z}_{2^a}^\times$.

5 Compute

$$\begin{aligned} P_B, Q_B &= [\omega_2]\psi(P_0), [\omega_2^{-1}]\psi(Q_0), \\ P_{AB}, Q_{AB} &= [\omega_2]\psi''(P_A), [\omega_2^{-1}]\psi''(Q_A). \end{aligned}$$

6 Set $x' =$

$$\begin{aligned} E_0, (P_0, Q_0), (R_0, S_0), E_{A_1}, (R_{A_1}, S_{A_1}), E_A, (R_A, S_A), (P_A, Q_A), \\ E_B, (P_B, Q_B), E_{AB}, (P_{AB}, Q_{AB}). \end{aligned}$$

7 **return** $(\psi, \psi', \psi'', x')$.If $a \neq 0$, then

$$c\tilde{M}_2 = b(\text{tr}(\tilde{M}_1)I - M_1),$$

and therefore

$$\ker(\tilde{M}_2) = \ker(\text{tr}(\tilde{M}_1)I - M_1).$$

Consequently, there are at most two possible kernels that $\tilde{M}_2$ can have among the four cyclic subgroups of $\mathbb{Z}_3 \times \mathbb{Z}_3$.

Since $\ker(M_1)$ and $\ker(M_2)$ are uniformly random cyclic subgroups of $\mathbb{Z}_{3^b} \times \mathbb{Z}_{3^b}$, the probability that $\ker(\tilde{M}_2)$ does not belong to these two possibilities is at least $1/2$. In this case, the subring generated by $\{1, \sigma_1, \sigma_2\}$ has rank strictly greater than 2.

Finally, since there is no subring of rank 3 in a quaternion algebra, the generated subring must have full rank. $\square$

Theorem 6 (IsERP, Theorem 3 in [11](heuristic)). *Let $\tilde{E}, E$ be two super-singular elliptic curves over $\mathbb{F}_{p^2}$ and let $\varphi : \tilde{E} \rightarrow E$ be an isogeny of degree N for some integer N . Given $\text{End}(\tilde{E})$ and an efficient isogeny representation for φ , there exists a quantum polynomial algorithm that computes $\text{End}(E)$ in expected polynomial time in $\log p$ and $\log N$.*

Theorem 7 (SubOrderEndRing, Theorem 1 in [11](heuristic)). *Let E be a supersingular elliptic curve over $\mathbb{F}_{p^2}$. Given a full rank suborder $\mathcal{R}_E \subseteq \text{End}(E)$ of discriminant $\Delta_{\mathcal{R}}$, there exists a quantum algorithm that computes $\text{End}(E)$ in expected polynomial time in $\log p$ and $\log(\sqrt{\Delta_{\mathcal{R}}}/p)$.*

Remark 2. The heuristics involved in Theorem 6 and 7 are from the plausible heuristics of subroutines `RepresentInteger` and `GenStrongApproximation`.

Lemma 5.3 (heuristic). *Let $x \leftarrow \text{Recv}(1^\lambda)$. Given an efficient representation of a basis of $\text{End}(E_{A_1})$, there exists an algorithm that solves the *Recv* problem in expected polynomial time in λ .*

Proof. The algorithm $\mathcal{B}$ first computes a connecting ideal I [26, Algorithm 3.5] between the two endomorphism rings $\mathcal{O}_0 \cong \text{End}(E_0)$ and $\mathcal{O}_{A_1} \cong \text{End}(E_{A_1})$. Then, we search $\alpha \in I$ such that $\text{Nrd}(\alpha) \leq 2^a \cdot \text{Nrd}(I)$. Since the degree of ϕ_1 has the same size of $2^a \approx p^{1/3} \ll \sqrt{p}$, the expected number of such α is less than 1 according to the Gaussian Heuristic. We can expect that it is uniquely determined in most cases. When it is not unique, $\mathcal{B}$ can use the given torsion information $[\gamma]\phi_1(R_0)$ and $[\gamma]\phi_1(S_0)$ to verify the correctness of a candidate equivalent ideal $J = \frac{I \cdot \bar{\alpha}}{\text{Nrd}(I)}$. From the recovered degree $q = \text{Nrd}(J) = \text{Nrd}(\alpha)/\text{Nrd}(I)$, the adversary $\mathcal{B}$ recovers the masking values $\gamma, \gamma' \in \mathbb{Z}_{3^b}^\times$ such that $(R_{A_1}, S_{A_1}) = ([\gamma']\phi_1(R_0), [\gamma']\phi_1(S_0))$ and $(R_A, S_A) = ([\gamma]\phi(R_0), [\gamma]\phi(S_0))$ in the *Recv* problem. Finally, by computing four dimensional isogenies, $\mathcal{B}$ recovers the two-dimensional isogeny Φ from the recovered 3^b -torsion information in the similar way to the Castryck-Decru attack. $\square$

Theorem 8. *For any PPT algebraic adversary $\mathcal{A}$ that solves the C-INKE problem, there exists a quantum adversary $\mathcal{B}$ that solves the *Recv* problem or the *Send* problem in time $O(1/\epsilon)$ where $\epsilon = \text{Adv}^{\text{C-INKE}}(\mathcal{A})$.*

Proof. Our proof follows the proof technique in [2, Theorem 6.7]. Let $\mathcal{A}$ be an adversary against C-INKE and it produces a curve $E_{A_1 B}$. Since the adversary is algebraic, it must explain its output $E_{A_1 B}$ with one of five possible explanations:

1. An isogeny $\varphi_{\text{expl}} : E_0 \rightarrow E_{A_1 B}$, or
2. An isogeny $\varphi_{\text{expl}} : E_{A_1} \rightarrow E_{A_1 B}$,
3. An isogeny $\varphi_{\text{expl}} : E_A \rightarrow E_{A_1 B}$,
4. An isogeny $\varphi_{\text{expl}} : E_B \rightarrow E_{A_1 B}$,
5. An isogeny $\varphi_{\text{expl}} : E_{AB} \rightarrow E_{A_1 B}$.

We construct an adversary $\mathcal{B}$ (Algorithm 22 in Appendix C) that solves one of the two problems *Recv*, and *Send* by guessing the type of the explanation the adversary $\mathcal{A}$ will produce. If the guess is incorrect, $\mathcal{B}$ aborts. Algorithm $\mathcal{B}$ solves the problems as follows:

- **Case 1.** ($\varphi_{\text{expl}} : E_0 \rightarrow E_{A_1 B}$) If the isogeny φ_{expl} will be from E_0 to $E_{A_1 B}$, $\mathcal{B}$ will solve the *Recv* problem as follows. From an instance $x \leftarrow \text{Recv}(1^\lambda)$,

$\mathcal{B}$ computes $(\psi, \psi', \psi'', y) \leftarrow \text{ExtendRecv}(x)$. $\mathcal{B}$ runs $\mathcal{A}$ with an input y . If $\mathcal{A}$ solves the C-INKE problem, it outputs $E_{A_1 B}$ and an explanation φ_{expl} . Since $\mathcal{B}$ knows an isogeny path $\widehat{\psi}' \circ \varphi_{\text{expl}} : E_0 \rightarrow E_{A_1}$, by Theorem 6, $\mathcal{B}$ can compute $\text{End}(E_{A_1})$. By Lemma 5.3, $\mathcal{B}$ can solve the Recv problem.

- **Case 2.** ($\varphi_{\text{expl}} : E_{A_1} \rightarrow E_{A_1 B}$) If the isogeny φ_{expl} will be from E_{A_1} to $E_{A_1 B}$, $\mathcal{B}$ will solve the Recv problem or the Send problem as follows.

Case 2-1. If $\text{ExtractSendExtractSend}((s_{\varphi_{\text{expl}}}, \mathcal{E}), x)$ will not abort, $\mathcal{B}$ will solve the Send problem. Given an instance $x \leftarrow \text{C-INKE}(1^\lambda)$, $\mathcal{B}$ runs

$$\begin{aligned} (E_{A_1 B}, (s_{\varphi_{\text{expl}}}, \mathcal{E})) &\leftarrow \mathcal{A}(x) \\ \psi &\leftarrow \text{ExtractSend}((s_{\varphi_{\text{expl}}}, \mathcal{E}), x). \end{aligned}$$

$\mathcal{B}$ outputs ψ as a solution of the Send problem.

Case 2-2. If $\text{ExtractSendExtractSend}((s_{\varphi_{\text{expl}}}, \mathcal{E}), x)$ will abort, $\mathcal{B}$ uses Lemma 5.2 to obtain a full-rank suborder of $\text{End}(E_{A_1})$. Given an instance $x \leftarrow \text{Recv}(1^\lambda)$, $\mathcal{B}$ computes

$$\begin{aligned} (\psi_1, \psi'_1, \psi''_1, y_1) &\leftarrow \text{ExtendRecv}(x) \\ (\psi_2, \psi'_2, \psi''_2, y_2) &\leftarrow \text{ExtendRecv}(x). \end{aligned}$$

By running

$$\begin{aligned} (E_{1, A_1 B}, \varphi_{1, \text{expl}}) &\leftarrow \mathcal{A}(y_1) \\ (E_{2, A_1 B}, \varphi_{2, \text{expl}}) &\leftarrow \mathcal{A}(y_2), \end{aligned}$$

$\mathcal{B}$ obtains a subring of $\text{End}(E_{A_1})$ generated by $\{1, \widehat{\varphi}_{1, \text{expl}} \circ \psi'_1, \widehat{\varphi}_{2, \text{expl}} \circ \psi'_2\}$. With probability at least $1/2$, the subring has rank 4. By using Theorem 7, $\mathcal{B}$ can compute $\text{End}(E_{A_1})$ from the full-rank suborder. According to Lemma 5.3, $\mathcal{B}$ can solve the Recv problem from the $\text{End}(E_{A_1})$.

- **Case 3.** ($\varphi_{\text{expl}} : E_A \rightarrow E_{A_1 B}$) If the isogeny φ_{expl} will be from E_A to $E_{A_1 B}$, $\mathcal{B}$ will solve the Send problem or the Recv problem by using the similar way of Case 2.

Case 3-1. If $\text{ExtractSendExtractSend}((s_{\varphi_{\text{expl}}}, \mathcal{E}), x)$ will not abort, $\mathcal{B}$ will solve the Send problem. Given an instance $x \leftarrow \text{C-INKE}(1^\lambda)$, $\mathcal{B}$ runs

$$\begin{aligned} (E_{A_1 B}, (s_{\varphi_{\text{expl}}}, \mathcal{E})) &\leftarrow \mathcal{A}(x) \\ \psi &\leftarrow \text{ExtractSend}((s_{\varphi_{\text{expl}}}, \mathcal{E}), x). \end{aligned}$$

$\mathcal{B}$ outputs ψ as a solution of the Send problem.

Case 3-2. If $\text{ExtractSendExtractSend}((s_{\varphi_{\text{expl}}}, \mathcal{E}), x)$ will abort, $\mathcal{B}$ will solve the Recv problem. Given an instance $x \leftarrow \text{Recv}(1^\lambda)$, $\mathcal{B}$ computes

$$\begin{aligned} (\psi_1, \psi'_1, \psi''_1, y_1) &\leftarrow \text{ExtendRecv}(x) \\ (\psi_2, \psi'_2, \psi''_2, y_2) &\leftarrow \text{ExtendRecv}(x) \\ (\psi_3, \psi'_3, \psi''_3, y_3) &\leftarrow \text{ExtendRecv}(x). \end{aligned}$$

By running

$$\begin{aligned}(E_{1,A_1B}, \varphi_{1,\text{expl}}) &\leftarrow \mathcal{A}(y_1) \\ (E_{2,A_1B}, \varphi_{2,\text{expl}}) &\leftarrow \mathcal{A}(y_2) \\ (E_{3,A_1B}, \varphi_{3,\text{expl}}) &\leftarrow \mathcal{A}(y_3),\end{aligned}$$

$\mathcal{B}$ obtains a subring of $\text{End}(E_{A_1})$ generated by $\{1, \widehat{\sigma}_2 \circ \sigma_1, \widehat{\sigma}_3 \circ \sigma_1\}$ where $\sigma_i = \widehat{\varphi}_{i,\text{expl}} \circ \psi'_i : E_{A_1} \rightarrow E_A$. By Lemma 5.2, with the probability at least $1/2$, the subring has rank 4. By using Theorem 7, $\mathcal{B}$ can compute $\text{End}(E_{A_1})$ from the full-rank suborder. By Lemma 5.3, $\mathcal{B}$ can solve the Recv problem from the $\text{End}(E_{A_1})$.

- **Case 4.** ($\varphi_{\text{expl}} : E_B \rightarrow E_{A_1B}$) If the isogeny φ_{expl} will be from E_B to E_{A_1B} , $\mathcal{B}$ will solve the Recv problem as follows. Given an instance $x \leftarrow \text{Recv}(1^\lambda)$, $\mathcal{B}$ computes $(\psi, \psi', \psi'', y) \leftarrow \text{ExtendRecv}(x)$. By running $(E_{A_1B}, \varphi_{\text{expl}}) \leftarrow \mathcal{A}(x)$, $\mathcal{B}$ obtains an isogeny path $\sigma = \widehat{\psi}' \circ \varphi_{\text{expl}} \circ \psi : E_0 \rightarrow E_{A_1}$. By Theorem 6, $\mathcal{B}$ obtains $\text{End}(E_{A_1})$ from the isogeny path σ . By Lemma 5.3, $\mathcal{B}$ computes a solution of the Recv problem from the endomorphism ring $\text{End}(E_{A_1})$.
- **Case 5.** ($\varphi_{\text{expl}} : E_{AB} \rightarrow E_{A_1B}$) If the isogeny φ_{expl} will be from E_{AB} to E_{A_1B} , $\mathcal{B}$ will solve the Send problem or the Recv problem.

Case 5-1. If $\text{ExtractSendExtractSend}((s_{\varphi_{\text{expl}}}, \mathcal{E}), x)$ will not abort, $\mathcal{B}$ will solve the Send problem. Given an instance $x \leftarrow \text{C-INKE}(1^\lambda)$, $\mathcal{B}$ runs

$$\begin{aligned}(E_{A_1B}, (s_{\varphi_{\text{expl}}}, \mathcal{E})) &\leftarrow \mathcal{A}(x) \\ \psi &\leftarrow \text{ExtractSend}((s_{\varphi_{\text{expl}}}, \mathcal{E}), x).\end{aligned}$$

$\mathcal{B}$ outputs ψ as a solution of the Send problem.

Case 5-2. If $\text{ExtractSendExtractSend}((s_{\varphi_{\text{expl}}}, \mathcal{E}), x)$ will abort, $\mathcal{B}$ will solve the Recv problem. Given an instance $x \leftarrow \text{Recv}(1^\lambda)$, $\mathcal{B}$ computes

$$\begin{aligned}(\psi_1, \psi'_1, \psi''_1, y_1) &\leftarrow \text{ExtendRecv}(x) \\ (\psi_2, \psi'_2, \psi''_2, y_2) &\leftarrow \text{ExtendRecv}(x) \\ (\psi_3, \psi'_3, \psi''_3, y_3) &\leftarrow \text{ExtendRecv}(x).\end{aligned}$$

By running

$$\begin{aligned}(E_{1,A_1B}, \varphi_{1,\text{expl}}) &\leftarrow \mathcal{A}(y_1) \\ (E_{2,A_1B}, \varphi_{2,\text{expl}}) &\leftarrow \mathcal{A}(y_2) \\ (E_{3,A_1B}, \varphi_{3,\text{expl}}) &\leftarrow \mathcal{A}(y_3),\end{aligned}$$

$\mathcal{B}$ obtains a subring of $\text{End}(E_{A_1})$ generated by $\{1, \widehat{\sigma}_2 \circ \sigma_1, \widehat{\sigma}_3 \circ \sigma_1\}$ where $\sigma_i = \widehat{\psi}'' \circ \widehat{\varphi}_{i,\text{expl}} \circ \psi'_i : E_{A_1} \rightarrow E_A$. By Lemma 5.2, with the probability at least $1/2$, the subring has rank 4. By using Theorem 7, $\mathcal{B}$ can compute $\text{End}(E_{A_1})$ from the full-rank suborder. According to Lemma 5.3, $\mathcal{B}$ can solve the Recv problem by using the $\text{End}(E_{A_1})$.

As the guess of $\mathcal{B}$ is independent of $\mathcal{A}$, the security loss is a constant factor 8. In Cases 2-2, 3-2, and 5-2, the algorithm $\mathcal{B}$ collects non-scalar endomorphisms in $\text{End}(E_{A_1})$ by extracting either explicit non-scalar endomorphisms or isogeny paths from E_{A_1} to E_A . Since the consecutive successful executions of A reduces the success probability quadratically or cubically, we use a slightly different approach in the actual algorithm (Algorithm 22).

To ensure that the running time of $\mathcal{B}$ remains linear in $1/\varepsilon$, the algorithm is designed to repeat the overall procedure multiple times while storing any newly obtaining non-scalar endomorphisms of E_{A_1} . In Case 2-2, a non-scalar endomorphism can be obtained directly and stored in the table. In contrast, in Cases 3-2 and 5-2, the algorithm only recovers an isogeny path from E_{A_1} to E_A . In these cases, once such a path is obtained, it is combined with a previously known path to derive a non-scalar endomorphism, which is then stored.

□

5.4 Comparison with the security of POKÉ

We emphasize that there is currently no formal reduction showing that the receiver security of POKÉ is stronger than that of INKE, nor vice versa. Although no direct reduction between the two schemes is known, for receiver security, INKE exhibits less information leakage than POKÉ with respect to all currently known attacks.

If the sender's security is defined following the approach of POKÉ, one may argue that INKE exposes less information about the sender's isogeny, since it does not reveal the 5^c -torsion information. However, as discussed in Section 5.2, in both INKE and POKÉ, the sender's security inherently depends on the receiver's security. Taking this property into account, the 5^c -torsion information in POKÉ can be viewed as corresponding to the intermediate curve and associated 3^b -torsion information in INKE. Currently, it remains unclear which of these reveals more information, and a precise comparison between the two forms of leakage is not yet well understood.

It is also worth noting that POKÉ does not admit a security proof in the AIM, since POKÉ uses points as shared secrets. Any attempt to prove the security of POKÉ within an algebraic framework would therefore require assuming a stronger model in which an object consisting of a curve together with a chosen basis is treated as a single algebraic entity like a level structure.

In summary, INKE is not only more efficient than POKÉ, but also at least as secure against all known attacks. Moreover, INKE enjoys additional structural advantages: the security of INKE can be analyzed within a well-defined algebraic model, which shows a security reduction from the security of the sender and the receiver. These properties provide a clearer and more robust foundation for the security of INKE compared to POKÉ.

6 Implementation Results

This section presents the implementation results of INKE. We first present the parameters of INKE for NIST security levels 1, 3, and 5, and compare their sizes with those of POKÉ. Subsequently, we provide a performance comparison between the two schemes.

6.1 Parameter selection

This section explains the parameter set used for the implementations, targeting NIST security levels 1, 3, and 5. Rather than directly presenting the key sizes, we include the components of the public key (**pk**), secret key (**sk**), and ciphertext (**ct**) in Table 1 to clarify how these sizes are derived for both POKÉ and INKE.

Table 1: Comparison of components of public key, secret key, and ciphertext between POKÉ and INKE.

	POKÉ	INKE
pk	$E_A, P_A, Q_A, R_A, S_A, X_A, Y_A$	$E_A, P_A, Q_A, R_A, S_A, E_{A_1}, R_{A_1}, S_{A_1}$
sk	$q, \alpha_2, \beta_2, \delta_5$	q, α_2, β_2
ct	$(E_B, P_B, Q_B), (E_{AB}, P_{AB}, Q_{AB}), \text{ct}'$	$(E_B, P_B, Q_B), (E_{AB}, P_{AB}, Q_{AB}), \text{ct}'$

In general, for a security parameter λ , the parameters a and b are chosen such that $a \approx \lambda$ and $3^b \approx 2^{2\lambda}$, resulting in a prime size of approximately 3λ bits. In comparison, the prime size required for POKÉ is about $10\lambda/3$, meaning that our scheme achieves a reduction of roughly $\lambda/3$ bits in the prime size. Table 2 compares the public-key and ciphertext sizes of INKE, POKÉ and PIKE at the 128-, 192-, and 256-bit security levels. We provide the INKE primes in NIST-I, NIST-III and NIST-V security levels as follows.

NIST-I

$$p = 2^{128} \cdot 3^{162} \cdot 127 - 1, \log_2(p) = 392$$

NIST-III

$$p = 2^{192} \cdot 3^{243} \cdot 5 \cdot 7 - 1, \log_2(p) = 583$$

NIST-V

$$p = 2^{257} \cdot 3^{324} \cdot 7^2 - 1, \log_2(p) = 777$$

As noted by the authors of POKÉ in [5], points of coprime order can be reduced into a single representative point by summing them. Since a POKÉ public key consists of one elliptic curve (of size $2 \log p$) and three sets of bases of coprime orders on the same curve, it can be represented using $6 \log p$ bits. The ciphertext, which consists of two elliptic curves and two sets of bases, can be represented using $12 \log p$ bits.

Additional compression is possible by expressing the basis with respect to a deterministic basis and using pairings. Under this compression, the public key can be reduced to $5 \log p$ bits, and the ciphertext to $4 \log p + 6a$ bits, where a is the exponent of 2 in $p + 1$.

Table 2: Comparison of public key and ciphertext sizes of INKE, POKÉ and PIKE on an Intel i7-9700 CPU. The sizes are measured in bytes.

Metric	NIST I			NIST III			NIST V		
	INKE	POKÉ	PIKE	INKE	POKÉ	PIKE	INKE	POKÉ	PIKE
$\log_2 p$	392	431	384	583	648	576	777	863	770
pk size (B)	588	324	480	874	486	720	1166	648	963
ct size (B)	588	648	576	874	972	864	1166	1296	1156
pk _{cmp} size (B)	440	270	272	656	405	408	873	540	545
ct _{cmp} size (B)	292	312	400	436	468	600	581	624	802

Remark 3. The ciphertext size of POKÉ is in fact larger. In the POKÉ encryption, the points X_B and Y_B are masked by a random invertible matrix. In principle, one could reduce the ciphertext size by instead choosing X_B, Y_B as a canonical basis of $E_B[5^c]$, thereby omitting them from the ciphertext, which is mentioned in the POKÉ paper [5].

However, this optimization comes at the cost of increased encryption time. In particular, it requires solving a discrete logarithm problem to recover the masking matrix D_5 satisfying

$$[X_B, Y_B]^T = D_5[\psi(X_0), \psi(Y_0)]^T,$$

and then computing the corresponding

$$[X_{AB}, Y_{AB}]^T = D_5[\psi'(X_A), \psi'(Y_A)]^T.$$

As a result, there is a clear trade-off between ciphertext size and encryption efficiency. Although choosing a canonical basis for X_B, Y_B reduces the ciphertext size, it incurs additional computational overhead due to the required discrete logarithm computation, leading to slower encryption. For a fair comparison, we therefore do not adopt this optimization in our performance evaluation.

Remark 4. Since PIKE and INKE impose the same constraints on the prime, INKE can also use the prime proposed in PIKE. If the prime suggested in PIKE is used in the INKE scheme, the size of the prime p and the uncompressed ciphertext size in Table 2 become identical for INKE and PIKE. By using the PIKE prime, INKE becomes faster in overall performance due to the reduced prime size, while the use of some 5-isogenies instead of 3-isogenies in the encryption procedure may introduce a slight slowdown.

Remark 5. As mentioned in the POKÉ and PIKE papers, it is possible to use BSIDH-like primes to speed up the decryption. However, it should be noted that the presence of distinct small factors reduces the search space of q due to the following condition:

$$\gcd(q(2^a - q), 2 \cdot 3 \cdot C) = 1.$$

When a is even and $C = 1$ (the case of INKE), this condition implies that $q \equiv 5 \pmod{6}$, reducing the search space by a factor of 6. When $C = 5$ (the cases of POKÉ and PIKE), the search space is reduced by a factor of 10, which results in approximately 0.74-bit loss in security compared to INKE, as discussed in Section 5.1. If $C = 5 \cdot 7 \cdot 11 \cdot 13 \cdot 17$, the search space is reduced by $\frac{135}{3094}$, which results in approximately 1.93-bit loss in security compared to INKE. This loss in security should be compensated by increasing the exponent a .

In our scheme, the public key consists of two elliptic curves and associated bases: $(E_A, (P_A, Q_A), (R_A, S_A))$ and $(E_{A_1}, (R_{A_1}, S_{A_1}))$. Without compression, this can be represented by $12 \log p$ bits. Using compression, the size is reduced to $7 \log p + 3b \log 3 \approx 9 \log p$, where b is the exponent of 3 in $p+1$. The ciphertext is structurally identical to that of POKÉ, and is thus represented by $12 \log p$ bits uncompressed, or $4 \log p + 6a \approx 6 \log p$ bits with compression.

To summarize, the size changes are as follows:

$$\begin{aligned} \text{pk} : 6 \log p \approx 20\lambda &\rightarrow 12 \log p' \approx 36\lambda \text{ (80\% increase)} \\ \text{ct} : 12 \log p \approx 40\lambda &\rightarrow 12 \log p' \approx 36\lambda \text{ (10\% decrease)} \\ \text{pk}_{\text{cmp}} : 5 \log p \approx \frac{50}{3}\lambda &\rightarrow 7 \log p' + 3b \log 3 \approx 27\lambda \text{ (60\% increase)} \\ \text{ct}_{\text{cmp}} : 4 \log p + 6a \approx \frac{58\lambda}{3} &\rightarrow 4 \log p' + 6a \approx 18\lambda \text{ (6.9\% decrease)} \end{aligned}$$

The public key in our scheme is approximately 80% larger than that of POKÉ, while the ciphertext is about 10% smaller. When compression is applied, we observe a 60% increase in public key size and a 6.9% reduction in ciphertext size compared to POKÉ.

6.2 Performance analysis

For performance analysis, we implemented the uncompressed version of POKÉ and INKE in C by modifying SQISign2D's reference code⁴, which implements the (2,2)-isogeny computation between abelian surfaces proposed in [14]. It uses multi-precision integers and quaternion algebras built upon the GMP library⁵ and the implementation of the arithmetic in the finite field $\mathbb{F}_p$ is generated by

⁴ <https://github.com/SQISign/sqisign2d-west-ac24>

⁵ <https://gmplib.org/>

Fiat-Crypto code generator [17]. We also use the PIKE reference code⁶ from [7] and optimize it for a fair comparison.

The CPU used for the measurements was an Intel i7-9700 Coffee Lake with a clock speed of 3 GHz, and the tests were conducted on an Ubuntu 24.04.3 LTS operating system using the clang 20.1.8 compiler. All benchmarks were executed with Intel Turbo Boost, EIST, and C-States disabled. Table 4 shows the performance comparison of POKÉ and INKE. All the estimated values are the average of 100 runs.

Table 3 shows a comparison between the SageMath and C implementations, confirming both the correctness and efficiency of our C implementation. In particular, our implementation consistently outperforms the reference SageMath code. The performance gap is larger for smaller parameter sets, which can be attributed to the higher impact of Python-level operations in SageMath. Our C implementation incorporates the following optimizations:

- Optimized finite field arithmetics based on [17].
- Precomputation of endomorphism actions on $E_0 : y^2 = x^3 + x$.
- Use of optimal strategies for 3-, 5-, and (2, 2)- isogenies.
- Optimized discrete logarithm computations using cubical arithmetic for pairings [36], [34].

Table 3: Comparison of our C implementation and the SageMath implementation of POKÉ [5] on an Intel i7-9700 CPU. The speedup shows our improvement over sage-POKÉ.

Level	SageMath (ms)			C (ms)			Improvement		
	KeyGen	Enc	Dec	KeyGen	Enc	Dec	KeyGen	Enc	Dec
NIST-I	1031	281	235	124	28	42	×8.28	×9.89	×5.62
NIST-III	2332	546	461	462	108	158	×5.05	×5.03	×2.92
NIST-V	4272	869	756	992	233	337	×4.31	×3.73	×2.24

The comparison between INKE and POKÉ is shown in Table 4. The relatively smaller improvement observed for the NIST level 1 can be explained by the choice of parameter in INKE. In particular, when selecting parameter of the form $2^{128} \cdot 3^{162} \cdot f - 1$ to obtain a prime modulus, the smallest admissible cofactor f turns out to be relatively large ($f = 127$). This reduces the performance improvement in the NIST level 1.

⁶ <https://github.com/Kaizhan-Lin/PIKE-C-Implementation>

Table 4: Comparison of C-implementations of POKÉ, PIKE and INKE on an Intel i7-9700 CPU. The runtimes are reported in Mcycles.

Level	INKE (Mcycles)			POKÉ (Mcycles)			PIKE (Mcycles)		
	KeyGen	Enc	Dec	KeyGen	Enc	Dec	KeyGen	Enc	Dec
NIST-I	296	80	60	359	83	122	372	71	65
NIST-III	953	261	195	1358	319	469	1271	250	222
NIST-V	2096	572	426	2950	685	997	2760	543	480

As shown in Table 4, by reducing the finite field size, INKE shows improvements in overall speeds. While it is apparent that INKE surpasses POKÉ in decryption, a more detailed analysis is required to fully clarify the comparison in encryption. Although INKE encryption involves one additional 3^b -isogeny codomain computation, it remains slightly faster than POKÉ in encryption, because POKÉ requires the evaluation of two 5^c -torsion bases. The underlying reason for this is analyzed in Appendix A. In particular, at the 128-bit security level, INKE achieves speedups of $\times 1.21$ in key generation, $\times 1.03$ in encryption, and $\times 2.03$ in decryption. Although the proposed method nearly doubles the public key size, the ciphertext size is reduced compared to that of POKÉ. Notably, the significant gain in decryption efficiency is a major advantage, especially for applications such as real-time data processing, network communication, and database encryption, where decryption speed is critical to overall system performance and user experience.

We also provide C implementations of the KEM version of INKE, POKÉ and PIKE, obtained by applying the Fujisaki-Okamoto transform. In general, the decapsulation time is given by the sum of the encryption and decryption times. However, if we adapt the simplified re-encryption technique in SIKE KEM [22] to our setting, the decapsulation can be performed with the additional cost of one 3^b -isogeny computation, two point evaluations (actually three point evaluations because of the x-only representation).

This optimization leverages the fact that the decryption rejects whenever the two-dimensional isogeny does not split, and that, once P_B and Q_B are fixed, the kernel representation of the two-dimensional isogeny is uniquely determined. Using decryption as a validation check relies on the hardness of finding a splitting isogeny from a given product of elliptic curves. Algorithm 11 shows the optimized decapsulation of INKE. Algorithm 12 and 13 show the optimized decapsulation algorithms for POKÉ and PIKE using the same method. We highlight in yellow the components that are not in INKE.

Algorithm 11: Decapsulation (INKE)

Input: $s, \text{sk} = (q, \alpha_2, \beta_2)$, $\text{ct} = ((E_B, P_B, Q_B), (E_{AB}, P_{AB}, Q_{AB}), \text{ct}')$
Output: A shared key K

```

1  $m' \leftarrow \text{Dec}(\text{sk}, \text{ct})$ 
2 if  $m' = \perp$  then
3    $K \leftarrow H(s || \text{ct})$ 
4   return  $K$ .
5  $(r', \omega'_2) \leftarrow G(m' || \text{pk})$ 
6  $E'_B, P'_B, Q'_B \leftarrow \text{isogen}_{3^b}(P_0, Q_0; r')$ 
7 if  $j(E'_B) = j(E_B)$  and  $P_B = [\omega'_2]P'_B$  and  $Q_B = [1/\omega'_2]Q'_B$  then
8    $K \leftarrow H(m' || \text{ct})$ 
9 else
10   $K \leftarrow H(s || \text{ct})$ 
11 return  $K$ .
```

Algorithm 12: Decapsulation (POKÉ)

Input: $s, \text{sk} = (q, \alpha_2, \beta_2)$,
 $\text{ct} = ((E_B, P_B, Q_B, X_B, Y_B), (E_{AB}, P_{AB}, Q_{AB}), \text{ct}')$
Output: A shared key K

```

1  $m' \leftarrow \text{Dec}(\text{sk}, \text{ct})$ 
2 if  $m' = \perp$  then
3    $K \leftarrow H(s || \text{ct})$ 
4   return  $K$ .
5  $(r', \omega'_2, D_5) \leftarrow G(m' || \text{pk})$ 
6  $E'_B, P'_B, Q'_B, X'_B, Y'_B \leftarrow \text{isogen}_{3^b}(P_0, Q_0, X_0, Y_0; r')$ 
7 if  $j(E'_B) = j(E_B)$  and  $P_B = [\omega'_2]P'_B$  and  $Q_B = [1/\omega'_2]Q'_B$  and  

    $[X_B, Y_B]^T = D_5[X'_B, Y'_B]^T$  then
8    $K \leftarrow H(m' || \text{ct})$ 
9 else
10   $K \leftarrow H(s || \text{ct})$ 
11 return  $K$ .
```

G and H are hash functions, and $\text{isogen}_C(P; r')$ is a deterministic C -isogeny computation and evaluation algorithm based on the kernel $\langle R_0 + [r']S_0 \rangle$ for the canonical basis $R_0, S_0 \in E_0[C]$. In the case of PIKE, we have $C = 3^b \cdot 5^c \approx 2^{2\lambda}$. Since the performance improvement of INKE over POKÉ is most pronounced in decryption, optimizing the re-encryption step in decapsulation allows the KEM variant of INKE to achieve speedups comparable to those observed in decryption when compared with POKÉ. Table 5 shows the improvement in the optimized decapsulation.

Remark 6. Our optimized decapsulation algorithms prevents an IND-CCA adversary from manipulating the ciphertext by validating the codomain curve E_B and the evaluated points P_B and Q_B . However, for a given supersingular elliptic curve, there exist two distinct Montgomery representations that are isomorphic

Algorithm 13: Decapsulation (PIKE)

Input: $s, \text{sk} = (q, \alpha_2, \beta_2)$, $\text{ct} = ((E_B, P_B, Q_B, Y_B), (E_{AB}, P_{AB}, Q_{AB}, X_{AB}), \text{ct}')$
Output: A shared key K

```

1  $m' \leftarrow \text{Dec}(\text{sk}, \text{ct})$ 
2 if  $m' = \perp$  then
3    $K \leftarrow H(s || \text{ct})$ 
4   return  $K$ .
5  $(r'_1, r'_2, \omega'_2, \eta'_D, \zeta'_D) \leftarrow G(m' || \text{pk})$ 
6  $E'_{AB}, X'_{AB} \leftarrow \text{isogen}_C(X_B; r'_1)$ 
7  $E'_B, P'_B, Q'_B, Y'_B \leftarrow \text{isogen}_C(P_0, Q_0, Y_0; r'_2)$ 
8 if  $j(E'_B) = j(E_B)$  and  $P_B = [\omega'_2]P'_B$  and  $Q_B = [1/\omega'_2]Q'_B$  and  $X_{AB} = [\eta'_D]X'_{AB}$ 
   and  $Y_B = [\zeta'_D]Y'_B$  then
9    $K \leftarrow H(m' || \text{ct})$ 
10 else
11    $K \leftarrow H(s || \text{ct})$ 
12 return  $K$ .
```

to each other. The corresponding isomorphism is given by

$$\iota : (x, y) \mapsto (-x, iy).$$

This map transforms a Montgomery curve $E : y^2 = x^3 + Ax^2 + x$ into an isomorphic curve $\tilde{E} : y^2 = x^3 - Ax^2 + x$.

To prevent an adversary from exploiting this isomorphism to manipulate the ciphertext, the elliptic curve must be encoded in a canonical way. In particular, when performing binary encoding, one should deterministically select a unique representative by comparing A and $-A$ according to a fixed canonical ordering.

Table 5: Comparison of the decapsulation (Algorithm 11, 12 and 13) of INKE, POKÉ and PIKE. Timings are the average of 100 executions on an Intel i7-9700 CPU.

Level	Decapsulation (Mcycles)		
	INKE	POKÉ	PIKE
NIST-I	94	169	124
NIST-III	301	644	425
NIST-V	655	1400	927

7 Conclusion

In this paper, we propose INKE, a point-free variant of POKÉ with faster key generation, and decryption. Although POKÉ [5] is one of the most efficient

isogeny-based encryption schemes, it remains slower compared to other post-quantum cryptographic algorithms. To improve performance, we focus on reducing the size of the underlying prime at the same security level. Since this change affects the original structure of POKÉ, we adapt the scheme accordingly. By removing the reliance on point evaluation, INKE relaxes the smoothness constraints required in POKÉ, allowing the use of smaller primes. This leads to faster key generation, encryption, and decryption, making the scheme well-suited for practical applications. Furthermore, INKE does not require solving the discrete logarithm, contributing to its overall efficiency.

We further demonstrate that the security of INKE depends on the quantum hardness of key-recovery attacks against the receiver and the sender in the Algebraic Isogeny Model. INKE uses an intermediate curve as the shared secret instead of a 5^c -torsion basis, as in POKÉ. This structural feature not only improves performance but also enables a security reduction in the algebraic model. Our receiver's and sender's security are defined and analyzed analogously to those of POKÉ as both schemes share essentially the same structure. Although the receiver's security in INKE differs in that an intermediate curve is revealed, we show that this does not leak additional information with respect to known attacks. For sender security, we establish a reduction from a fixed instance of the CIST problem, following an approach analogous to that of POKÉ.

Although our design choice enlarges the public key size, our scheme slightly reduces the ciphertext size and offers notable improvements in key generation, encryption, and decryption, for all security levels. The use of a smaller finite field makes the proposed scheme more memory-friendly for embedded systems and particularly suitable for applications where fast decryption is critical.

Acknowledgements. We thank Gwanho Kim for insightful discussions and valuable comments on this work.

References

1. Aardal, M.A., Basso, A., De Feo, L., Patranabis, S., Wesolowski, B.: A complete security proof of sqsign. In: Tauman Kalai, Y., Kamara, S.F. (eds.) *Advances in Cryptology – CRYPTO 2025*. pp. 190–222. Springer Nature Switzerland, Cham (2025)
2. Aardal, M.A., Basso, A., Riepel, D.: The algebraic isogeny model: A general model with applications to sqsign and key exchanges. *Cryptology ePrint Archive* (2026)
3. Basso, A., Dartois, P., Feo, L.D., Leroux, A., Maino, L., Pope, G., Robert, D., Wesolowski, B.: SQIsign2D–west. In: Chung, K.M., Sasaki, Y. (eds.) *Advances in Cryptology – ASIACRYPT 2024*. pp. 339–370. Springer Nature Singapore, Singapore (2025). https://doi.org/10.1007/978-981-96-0891-1_11
4. Basso, A., Fouotsa, T.B.: New SIDH countermeasures for a more efficient key exchange. In: Guo, J., Steinfeld, R. (eds.) *Advances in Cryptology – ASIACRYPT 2023*. pp. 208–233. Springer Nature Singapore, Singapore (2023). https://doi.org/10.1007/978-981-99-8742-9_7

5. Basso, A., Maino, L.: Poké: A compact and efficient pke from higher-dimensional isogenies. In: Fehr, S., Fouque, P.A. (eds.) *Advances in Cryptology – EUROCRYPT 2025*. pp. 94–123. Springer Nature Switzerland, Cham (2025). https://doi.org/10.1007/978-3-031-91124-8_4
6. Basso, A., Maino, L., Pope, G.: FESTA: Fast encryption from supersingular torsion attacks. In: Guo, J., Steinfeld, R. (eds.) *Advances in Cryptology – ASIACRYPT 2023*. pp. 98–126. Springer Nature Singapore, Singapore (2023). https://doi.org/10.1007/978-981-99-8739-9_4
7. Cai, S., Chen, M., Lai, Y.F., Lin, K.: Pike: Faster isogeny-based public key encryption with pairing-assisted decryption. *Cryptology ePrint Archive* (2026)
8. Castryck, W., Decru, T.: An efficient key recovery attack on SIDH. In: Hazay, C., Stam, M. (eds.) *Advances in Cryptology – EUROCRYPT 2023*. pp. 423–447. Springer Nature Switzerland, Cham (2023). https://doi.org/10.1007/978-3-031-30589-4_15
9. Chen, M., Imran, M., Ivanyos, G., Kutas, P., Leroux, A., Petit, C.: Hidden stabilizers, the isogeny to endomorphism ring problem and the cryptanalysis of psidh. In: Guo, J., Steinfeld, R. (eds.) *Advances in Cryptology – ASIACRYPT 2023*. pp. 99–130. Springer Nature Singapore, Singapore (2023)
10. Chen, M., Leroux, A., Panny, L.: SCALLOP-HD: Group action from 2-dimensional isogenies. In: Tang, Q., Teague, V. (eds.) *Public-Key Cryptography – PKC 2024*. pp. 190–216. Springer Nature Switzerland, Cham (2024). https://doi.org/10.1007/978-3-031-57725-3_7
11. Chen, M., Petit, C.: Computing the endomorphism ring of a supersingular elliptic curve from a full rank suborder. In: Fehr, S., Fouque, P.A. (eds.) *Advances in Cryptology – EUROCRYPT 2025*. pp. 446–474. Springer Nature Switzerland, Cham (2025)
12. Childs, A., Jao, D., Soukharev, V.: Constructing elliptic curve isogenies in quantum subexponential time. *Journal of Mathematical Cryptology* **8**(1), 1–29 (2014)
13. Couveignes, J.M.: Hard homogeneous spaces. *IACR Cryptology ePrint Archive* **2006**, 291 (2006)
14. Dartois, P., Maino, L., Pope, G., Robert, D.: An algorithmic approach to (2, 2)-isogenies in the theta model and applications to isogeny-based cryptography. In: Chung, K.M., Sasaki, Y. (eds.) *Advances in Cryptology – ASIACRYPT 2024*. pp. 304–338. Springer Nature Singapore, Singapore (2025). https://doi.org/10.1007/978-981-96-0891-1_10
15. De Feo, L., Leroux, A., Longa, P., Wesolowski, B.: New algorithms for the deuring correspondence. In: Hazay, C., Stam, M. (eds.) *Advances in Cryptology – EUROCRYPT 2023*. pp. 659–690. Springer Nature Switzerland, Cham (2023)
16. Duparc, M., Fouotsa, T.B.: SQIPrime: A dimension 2 variant of SQISignHD with non-smooth challenge isogenies. In: Chung, K.M., Sasaki, Y. (eds.) *Advances in Cryptology – ASIACRYPT 2024*. pp. 396–429. Springer Nature Singapore, Singapore (2025). https://doi.org/10.1007/978-981-96-0891-1_13
17. Erbsen, A., Philipoom, J., Gross, J., Sloan, R., Chlipala, A.: Simple high-level code for cryptographic arithmetic: With proofs, without compromises. *ACM SIGOPS Operating Systems Review* **54**(1), 23–30 (2020). <https://doi.org/10.1145/3421473.3421477>
18. Fouotsa, T.B., Moriya, T., Petit, C.: M-SIDH and MD-SIDH: Countering SIDH attacks by masking information. In: Hazay, C., Stam, M. (eds.) *Advances in Cryptology – EUROCRYPT 2023*. pp. 282–309. Springer Nature Switzerland, Cham (2023). https://doi.org/10.1007/978-3-031-30589-4_10

19. Fujisaki, E., Okamoto, T.: Secure integration of asymmetric and symmetric encryption schemes. In: Wiener, M. (ed.) *Advances in Cryptology — CRYPTO’99*. pp. 537–554. Springer Berlin Heidelberg, Berlin, Heidelberg (1999). https://doi.org/10.1007/3-540-48405-1_34
20. Herlédan Le Merdy, A., Wesolowski, B.: The supersingular endomorphism ring problem given one endomorphism. *IACR Communications in Cryptology* **2**(1) (2025). <https://doi.org/10.62056/akgyivrzn>
21. Hofheinz, D., Hövelmanns, K., Kiltz, E.: A modular analysis of the Fujisaki-Okamoto transformation. In: *Theory of Cryptography Conference*. pp. 341–371. Springer (2017). https://doi.org/10.1007/978-3-319-70500-2_12
22. Jao, D., Azarderakhsh, R., Campagna, M., Costello, C., Feo, L.D., Hess, B., Jalali, A., Koziel, B., LaMacchia, B., Longa, P., Naehrig, M., Renes, J., Soukharev, V., Urbanik, D., Pereira, G., Karabina, K., Hutchinson, A.: Sike. Tech. rep., National Institute of Standards and Technology (2020), <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions>
23. Jao, D., De Feo, L.: Towards quantum-resistant cryptosystems from supersingular elliptic curve isogenies. In: Yang, B.Y. (ed.) *Post-Quantum Cryptography*. pp. 19–34. Springer Berlin Heidelberg, Berlin, Heidelberg (2011). https://doi.org/10.1007/978-3-642-25405-5_2
24. Kani, E.: The number of curves of genus two with elliptic differentials. *Journal für die reine und angewandte Mathematik* **485**, 93–122 (1997), <http://eudml.org/doc/183547>
25. Kiltz, E., Malone-Lee, J.: A general construction of IND-CCA2 secure public key encryption. In: *IMA International Conference on Cryptography and Coding*. pp. 152–166. Springer (2003). https://doi.org/10.1007/978-3-540-40974-8_13
26. Kirschmer, M., Voight, J.: Algorithmic enumeration of ideal classes for quaternion orders. *SIAM Journal on Computing* **39**(5), 1714–1747 (2010)
27. Maino, L., Martindale, C., Panny, L., Pope, G., Wesolowski, B.: A direct key recovery attack on SIDH. In: Hazay, C., Stam, M. (eds.) *Advances in Cryptology – EUROCRYPT 2023*. pp. 448–471. Springer Nature Switzerland, Cham (2023). https://doi.org/10.1007/978-3-031-30589-4_16
28. Moriya, T.: Is-cube: An isogeny-based compact kem using a boxed sidh diagram. *Cryptology ePrint Archive* (2023)
29. Moriya, T., Stopar, M.: Lit-sigamal: An efficient isogeny-based pke based on a lit diagram. In: Hanaoka, G., Yang, B.Y. (eds.) *Advances in Cryptology – ASIACRYPT 2025*. pp. 275–306. Springer Nature Singapore, Singapore (2026)
30. Mumford, D.: *Abelian varieties*, tata institute of fundamental research studies in mathematics, no. 5, tata institute of fundamental research, bombay, 1970. MR **282985**(44), 219
31. Nakagawa, K., Onuki, H.: QFESTA: Efficient algorithms and parameters for FESTA using quaternion algebras. In: Reyzin, L., Stebila, D. (eds.) *Advances in Cryptology – CRYPTO 2024*. pp. 75–106. Springer Nature Switzerland, Cham (2024). https://doi.org/10.1007/978-3-031-68388-6_4
32. Nakagawa, K., Onuki, H., Castryck, W., Chen, M., Invernizzi, R., Lorenzon, G., Vercauteren, F.: SQIsign2D-east: A new signature scheme using 2-dimensional isogenies. In: Chung, K.M., Sasaki, Y. (eds.) *Advances in Cryptology – ASIACRYPT 2024*. pp. 272–303. Springer Nature Singapore, Singapore (2025). https://doi.org/10.1007/978-981-96-0891-1_9
33. Panny, L., Petit, C., Stopar, M.: KLaPoTi: An asymptotically efficient isogeny group action from 2-dimensional isogenies. *Cryptology ePrint Archive* (2024)

34. Pope, G., Reijnders, K., Robert, D., Sferlazza, A., Smith, B.: Simpler and Faster Pairings from the Montgomery Ladder. *IACR Communications in Cryptology* **2025**(2) (Jul 2025). <https://doi.org/10.62056/ah2i893y6>, <https://inria.hal.science/hal-05142445>
35. Robert, D.: Breaking SIDH in polynomial time. In: Hazay, C., Stam, M. (eds.) *Advances in Cryptology – EUROCRYPT 2023*. pp. 472–503. Springer Nature Switzerland, Cham (2023). https://doi.org/10.1007/978-3-031-30589-4_17
36. Robert, D.: Fast pairings via biextensions and cubical arithmetic (Dec 2024), <https://hal.science/hal-04848028>, working paper or preprint
37. Silverman, J.H.: *The arithmetic of elliptic curves*, vol. 106. Springer (2009). <https://doi.org/10.1007/978-0-387-09494-6>

A Concrete Comparison of Encryption Speeds

Let us denote M, S and A the costs of a multiplication, a squaring, and an addition, respectively, in the finite field used by POKÉ. Similarly, let M', S', A' denote the corresponding costs in INKE. Since the prime size of INKE is approximately $\frac{9}{10} \times$ of that of POKÉ, we approximate

$$(M', S', A') \approx \frac{9}{10} \cdot (M, S, A).$$

A 3^b -isogeny codomain computation consists of the operations xISOG, xEVAL, and xTPL (see Appendix B). As $b \approx \log_3 2 \cdot \lambda$, it follows from Algorithm 20 that the total cost of a single 3^b -isogeny computation is

$$\begin{aligned} & b \times (\text{xISOG} + c_1 \cdot \text{xEVAL} + c_2 \cdot \text{xTPL}) \\ &= b \cdot (2M + 3S + 12A + c_1 \cdot (4M + 2S + 4A) + c_2 \cdot (7M + 5S + 10A)) \\ &\lesssim \lambda \cdot \log_3 2 \cdot (31.5M + 20.8S + 45.4A) \end{aligned}$$

where xISOG costs $2M + 3S + 12A$, xEVAL costs $4M + 2S + 4A$, and xTPL costs $7M + 5S + 10A$. The factors c_1 and c_2 depend on the optimal strategy for the 3^b -isogeny computation. Using dynamic programming as in [23], we obtain $4 \leq c_1 \leq 5$ and $3 \leq c_2 \leq 4$ for $\lambda \in \{128, 192, 256\}$. A comparison of the isogeny computation costs is presented in Table 6.

On the other hand, computing $x([a]P + [b]Q)$ from the points $(x(P), x(Q), x(P - Q))$ consists of operations xDBL and xADD (see Appendix B) and it costs

$$\begin{aligned} & \log_2 N \cdot (\text{xDBL} + \text{xADD}) \\ &= \log_2 N \cdot (8M + 4S + 13A), \end{aligned}$$

where xDBL costs $4M + 2S + 7A$, xADD costs $4M + 2S + 6A$ and N is the order of the given points P, Q . If we combine these two operations into xDBLADD in Appendix B, the cost reduces to $7M + 4S + 8A$. The comparison of the scalar multiplications are illustrated in Table 7.

In conclusion, the POKÉ encryption requires approximately $12.7M + 4.1S + 5.6A$ more operations than that of INKE. Considering the total encryption cost of the POKÉ encryption is approximately $149.4M + 88.8S + 185.1A$, the difference is about 8.5% of the total cost of the POKÉ encryption.

	POKE encryption	INKE encryption
Operations	the isogeny $\psi : E_0 \rightarrow E_B$ computation and the evaluation of two bases $(P_0, Q_0), (X_0, Y_0)$ the isogeny $\psi' : E_A \rightarrow E_{AB}$ computation and the evaluation of two bases $(P_A, Q_A), (X_A, Y_A)$	the isogeny $\psi : E_0 \rightarrow E_B$ computation and the evaluation of one basis (P_0, Q_0) the isogeny $\psi' : E_{A_1} \rightarrow E_{A_1B}$ computation the isogeny $\psi'' : E_A \rightarrow E_{AB}$ computation and the evaluation of one basis (P_A, Q_A)
Cost	$2\lambda \cdot \log_3 2 \cdot (31.5M + 20.8S + 45.4A) + 12\lambda \cdot \log_3 2 \cdot (4M + 2S + 4A)$	$3\lambda \cdot \log_3 2 \cdot (31.5M' + 20.8S' + 45.4A') + 6\lambda \cdot \log_3 2 \cdot (4M' + 2S' + 4A')$
Difference	$\text{INKE} - \text{POKE} = \lambda \cdot (5.4M + 6.2S + 15.1A)$	

Table 6: Comparison of isogeny codomain computations and point evaluations

	POKE encryption	INKE encryption
Operations	compute $[\omega_2]\psi(P_0), [1/\omega_2]\psi(Q_0)$ compute $[\omega_2]\psi'(P_A), [1/\omega_2]\psi'(Q_A)$ compute $D_5[\psi(X_0), \psi(Y_0)]^T$ compute $D_5[\psi'(X_A), \psi'(Y_A)]^T$	compute $[\omega_2]\psi(P_0), [1/\omega_2]\psi(Q_0)$ compute $[\omega_2]\psi''(P_A), [1/\omega_2]\psi''(Q_A)$
Cost	$6\lambda \cdot (7M + 4S + 8A) + 6 \cdot \lambda/3 \cdot (7M + 4S + 8A)$	$6\lambda \cdot (7M' + 4S' + 8A')$
Difference	$\text{INKE} - \text{POKE} = -\lambda \cdot (18.2M + 10.4S + 20.8A)$	

Table 7: Comparison of scalar multiplications

B Concrete Costs of Isogeny Computations and Scalar Multiplications

We demonstrate basic algorithms for isogeny and elliptic curve arithmetics used in Appendix A as follows.

Algorithm 14: $\text{xADD} = 4M + 2S + 6A$ **Input:** Three Montgomery points $P = (X_P : Z_P), Q = (X_Q : Z_Q), P - Q = (X_{PQ} : Z_{PQ})$ on the same elliptic curve**Output:** A Montgomery point $R = (X_R : Z_R) = P + Q$

- | | |
|-----------------------------------|---------------------------------------|
| 1. $t_0 \leftarrow X_P + Z_P$ | 7. $t_2 \leftarrow t_0 + t_1$ |
| 2. $t_1 \leftarrow X_P - Z_P$ | 8. $t_3 \leftarrow t_0 - t_1$ |
| 3. $t_2 \leftarrow X_Q + Z_Q$ | 9. $t_2 \leftarrow t_2^2$ |
| 4. $t_3 \leftarrow X_Q - Z_Q$ | 10. $t_3 \leftarrow t_3^2$ |
| 5. $t_0 \leftarrow t_0 \cdot t_3$ | 11. $t_2 \leftarrow Z_{PQ} \cdot t_2$ |
| 6. $t_1 \leftarrow t_1 \cdot t_2$ | 12. $Z_R \leftarrow X_{PQ} \cdot t_3$ |

Algorithm 15: $\text{xDBL} = 4M + 2S + 7A$ **Input:** A Montgomery point $P = (X_P : Z_P)$ and the elliptic curve $(A : C)$ in projective Montgomery form.**Output:** A Montgomery point $Q = (X_Q : Z_Q) = [2]P$

- | | |
|-----------------------------------|------------------------------------|
| 1. $t_0 \leftarrow t_0^2$ | 7. $t_1 \leftarrow t_1 + t_1$ |
| 2. $t_1 \leftarrow X_P - Z_P$ | 8. $X_Q \leftarrow t_0 \cdot t_1$ |
| 3. $t_1 \leftarrow t_1^2$ | 9. $t_0 \leftarrow t_3 + A$ |
| 4. $t_2 \leftarrow t_0 - t_1$ | 10. $t_0 \leftarrow t_0 \cdot t_2$ |
| 5. $t_3 \leftarrow C + C$ | 11. $t_0 \leftarrow t_0 + t_1$ |
| 6. $t_1 \leftarrow t_1 \cdot t_3$ | 12. $Z_Q \leftarrow t_0 \cdot t_2$ |

Algorithm 16: $\text{xDBLADD} = 7M + 4S + 8A$ **Input:** Three Montgomery points $P = (X_P : Z_P), Q = (X_Q : Z_Q), P - Q = (X_{PQ} : Z_{PQ})$ and the elliptic curve $(A' : 1) = (A + 2C : 4C)$ from the projective Montgomery form $(A : C)$.**Output:** A Montgomery point $R = (X_R : Z_R) = [2]P$ and $S = (X_S : Z_S) = P + Q$.

- | | |
|------------------------------------|------------------------------------|
| 1. $t_0 \leftarrow X_P + Z_P$ | 11. $X_S \leftarrow A' \cdot t_2$ |
| 2. $t_1 \leftarrow X_P - Z_P$ | 12. $Z_S \leftarrow t_0 - t_1$ |
| 3. $X_R \leftarrow t_0^2$ | 13. $Z_R \leftarrow Z_R + X_S$ |
| 4. $t_2 \leftarrow X_Q - Z_Q$ | 14. $X_S \leftarrow t_0 + t_1$ |
| 5. $X_S \leftarrow X_Q + Z_Q$ | 15. $Z_R \leftarrow Z_R \cdot t_2$ |
| 6. $t_0 \leftarrow t_0 \cdot t_2$ | 16. $Z_S \leftarrow Z_S^2$ |
| 7. $Z_R \leftarrow t_1^2$ | 17. $X_S \leftarrow X_S^2$ |
| 8. $t_1 \leftarrow t_1 \cdot X_S$ | 18. $Z_S \leftarrow Z_S \cdot X_P$ |
| 9. $t_2 \leftarrow X_R - Z_R$ | 19. $X_S \leftarrow X_S \cdot Z_P$ |
| 10. $X_R \leftarrow X_R \cdot Z_R$ | |

Algorithm 17: $\times\text{TPL} = 7M + 5S + 10A$

Input: A Montgomery point $P = (X_P : Z_P)$ and the elliptic curve $(A' : C') = (A + 2C : A - 2C)$ in projective Montgomery form.

Output: A Montgomery point $Q = (X_Q : Z_Q) = [3]P$

- | | |
|------------------------------------|------------------------------------|
| 1. $t_0 \leftarrow X_P - Z_P$ | 12. $Z_Q \leftarrow t_2 \cdot C'$ |
| 2. $t_2 \leftarrow t_0^2$ | 13. $t_2 \leftarrow t_2 \cdot Z_Q$ |
| 3. $t_1 \leftarrow X_P + Z_P$ | 14. $t_3 \leftarrow t_2 - t_3$ |
| 4. $t_3 \leftarrow t_1^2$ | 15. $t_2 \leftarrow X_Q - Z_Q$ |
| 5. $t_4 \leftarrow t_1 + t_0$ | 16. $t_1 \leftarrow t_2 \cdot t_1$ |
| 6. $t_0 \leftarrow t_1 - t_0$ | 17. $t_2 \leftarrow t_3 + t_1$ |
| 7. $t_1 \leftarrow t_4^2$ | 18. $t_2 \leftarrow t_2^2$ |
| 8. $t_1 \leftarrow t_1 - t_3$ | 19. $X_Q \leftarrow t_2 \cdot t_4$ |
| 9. $t_1 \leftarrow t_1 - t_2$ | 20. $t_1 \leftarrow t_3 - t_1$ |
| 10. $X_Q \leftarrow t_3 \cdot A'$ | 21. $t_1 \leftarrow t_1^2$ |
| 11. $t_3 \leftarrow X_Q \cdot t_3$ | 22. $Z_Q \leftarrow t_1 \cdot t_0$ |

Algorithm 18: $\times\text{EVAL} = 4M + 2S + 4A$

Input: A 3-torsion kernel point $P = (Y_P : W_P)$ on a twisted Edward curve and a point $Q = (X_Q : Z_Q)$ to be evaluated.

Output: A Montgomery projective point $(X_R : Z_R)$

- | | |
|-----------------------------------|------------------------------------|
| 1. $t_0 \leftarrow X_Q + Z_Q$ | 6. $t_0 \leftarrow t_1 - t_0$ |
| 2. $t_1 \leftarrow X_Q - Z_Q$ | 7. $t_2 \leftarrow t_2^2$ |
| 3. $t_0 \leftarrow Y_P \cdot t_0$ | 8. $t_0 \leftarrow t_0^2$ |
| 4. $t_1 \leftarrow W_P \cdot t_1$ | 9. $X_R \leftarrow X_Q \cdot t_2$ |
| 5. $t_2 \leftarrow t_0 + t_1$ | 10. $Z_R \leftarrow Z_Q \cdot t_0$ |

Algorithm 19: $\times\text{SOG} = 2M + 3S + 12A$

Input: A 3-torsion kernel point $P = (Y_P : W_P)$ on a twisted Edward curve.

Output: The codomain curve $(A : C)$ in the projective Montgomery form.

- | | |
|-------------------------------|----------------------------------|
| 1. $t_0 \leftarrow Y_P^2$ | 10. $t_4 \leftarrow t_4 + t_4$ |
| 2. $t_1 \leftarrow W_P^2$ | 11. $t_4 \leftarrow t_1 + t_4$ |
| 3. $t_2 \leftarrow t_0 + t_1$ | 12. $C \leftarrow t_2 \cdot t_4$ |
| 4. $t_3 \leftarrow Y_P + W_P$ | 13. $t_4 \leftarrow t_1 + t_2$ |
| 5. $t_3 \leftarrow t_3^2$ | 14. $t_4 \leftarrow t_4 + t_4$ |
| 6. $t_3 \leftarrow t_3 - t_2$ | 15. $t_4 \leftarrow t_0 + t_4$ |
| 7. $t_2 \leftarrow t_1 + t_3$ | 16. $A \leftarrow t_3 \cdot t_4$ |
| 8. $t_3 \leftarrow t_3 + t_0$ | 17. $C \leftarrow A - C$ |
| 9. $t_4 \leftarrow t_3 + t_0$ | |

Algorithm 20: Velu's formula for a 3^b -isogeny

Input: A starting curve $E = (A : C)$, a 3^b -torsion kernel point $P = (X_P : Z_P)$ as Montgomery projective form, points P_i to be evaluated and the optimal 3^b -isogeny strategy **strategy3** as an array form.

Output: The image curve $E' = (A' : C')$ and the evaluated Montgomery projective points.

State: stack S , index stack I

- 1 Compute $A_{24} = A + 2C$ and $C_{24} = 4C$.
- 2 Compute $A_3 = A_{24}$ and $C_3 = A_{24} - C_{24}$.
- 3 Set $\text{index} = 0$ and $\text{st_index} = 0$.
- 4 **for** $\text{row} = 1$ **to** $b - 1$ **do**
- 5 **while** $\text{index} < b - \text{row}$ **do**
- 6 Push (P, index) onto (S, I)
- 7 Set $m \leftarrow \text{strategy3}[\text{st_index}]$ and $\text{st_index} \leftarrow \text{st_index} + 1$.
- 8 **for** $j = 1$ **to** m **do**
- 9 $P \leftarrow \text{xTPL}(P, (A_3, C_3))$
- 10 $\text{index} \leftarrow \text{index} + m$
- 11 Compute $K = (Y_K, W_K) = (X_P - Z_P, X_P + Z_P)$
- 12 Compute $(A_{24} : C_{24}) \leftarrow \text{xISOG}(K)$
- 13 Compute $A_3 = A_{24}$ and $C_3 = A_{24} - C_{24}$.
- 14 **for** $Q \in S$ **do**
- 15 $Q \leftarrow \text{xEVAL}(K, Q)$
- 16 **for** $i = 1$ **to** ℓ **do**
- 17 $P_i \leftarrow \text{xEVAL}(K, P_i)$
- 18 Pop (P, index) from (S, I)
- 19 Compute $K = (Y_K, W_K) = (X_R - Z_R, X_R + Z_R)$
- 20 Compute $(A_{24} : C_{24}) \leftarrow \text{xISOG}(K)$
- 21 **for** $i = 1$ **to** ℓ **do**
- 22 $P_i \leftarrow \text{xEVAL}(K, P_i)$
- 23 Compute $A' = 4 \cdot A_{24} - 2 \cdot C_{24}$ and $C' = C_{24}$
- 24 **return** $(E' = (A' : C'), \{P_i\})$

C The adversary $\mathcal{B}$ in Theorem 8

We introduce the quantum algorithm $\mathcal{B}_t$ used in the proof of Theorem 8. This algorithm repeats the entire procedure described in the proof of Theorem 8 $t = O(1/\varepsilon)$ times, where $\varepsilon = \text{Adv}^{\text{C-INKE}}(\mathcal{A})$. A single iteration of this procedure is described in Algorithm 21, which is invoked as a subroutine within $\mathcal{B}_t$. Note that a table $\mathcal{T}$ and a variable σ are used to store the previously obtained endomorphisms and an isogeny path.

Algorithm 21: Algorithm RecoverKey

Input: An instance $x \leftarrow \text{C-INKE}(1^\lambda)$.
Output: A solution of the Recv problem or the Send problem
State: A table $\mathcal{T}$ and a variable σ_{known} for solving the Recv problem.
Subroutine: An adversary $\mathcal{A}$ solving C-INKE

1 Parse x as
 $x_1 = (E_0, (P_0, Q_0), (R_0, S_0), E_{A_1}, (R_{A_1}, S_{A_1}), E_A, (P_A, Q_A), (R_A, S_A))$
and $x_2 = (E_B, (P_B, Q_B), E_{AB}, (P_{AB}, Q_{AB}))$.
2 Sample

$$(\text{expl}, \text{case}) \xleftarrow{\$} \{(0, -), (A_1, \text{good}), (A_1, \text{bad}), (A, \text{good}), (A, \text{bad}), (B, -), (AB, \text{good}), (AB, \text{bad})\}.$$

3 **if** $\text{expl} \in \{0, B\}$ **or** $\text{case} = \text{bad}$ **then**
4 **if** $\text{case} = \text{bad}$ **then**
5 Compute $(\psi, \psi', \psi'', y) \leftarrow \text{ExtendRecv}(x_1)$.
6 Compute $z \leftarrow \mathcal{A}(y)$.
7 **if** $z = \perp$ **then abort**.
8 Parse z as $(E_{A_1 B}, \varphi_{\text{expl}})$.
9 **if** $\text{domain}(\varphi_{\text{expl}}) \neq E_{\text{expl}}$ **then abort**
10 **if** $\text{expl} = A_1$ **then**
11 Store an endomorphism $\widehat{\varphi}_{\text{expl}} \circ \psi'$ into $\mathcal{T}$.
12 **else if** $\text{expl} = A$ **then**
13 **if** $\sigma_{\text{known}} = \perp$ **then**
14 Set $\sigma_{\text{known}} \leftarrow \widehat{\varphi}_{\text{expl}} \circ \psi'$.
15 **abort**
16 Store an endomorphism $\widehat{\sigma}_{\text{known}} \circ \widehat{\varphi}_{\text{expl}} \circ \psi'$ into $\mathcal{T}$.
17 **else if** $\text{expl} = AB$ **then**
18 **if** $\sigma_{\text{known}} = \perp$ **then**
19 Set $\sigma_{\text{known}} \leftarrow \widehat{\psi}'' \circ \widehat{\varphi}_{\text{expl}} \circ \psi'$.
20 **abort**
21 Store an endomorphism $\widehat{\sigma}_{\text{known}} \circ \widehat{\psi}'' \circ \widehat{\varphi}_{\text{expl}} \circ \psi'$ into $\mathcal{T}$.
22 Compute a subring $\mathcal{R}$ generated by $\mathcal{T}_{\text{expl}}$.
23 **if** $\text{rank}(\mathcal{R}) < 4$ **then abort**
24 Compute a subring $\mathcal{R}$ generated by $\mathcal{T}$.
25 Compute $\text{End}(E_{A_1})$ from $\mathcal{R}$. // Theorem 7
26 Compute a solution ϕ of the Recv problem from $\text{End}(E_{A_1})$.
 // Lemma 5.3
27 **return** ϕ as a solution of the Recv problem.
28 **else if** $\text{case} = \text{good}$ **then**
29 Compute $z \leftarrow \mathcal{A}(x)$.
30 **if** $z = \perp$ **then abort**.
31 Parse z as $(E_{A_1 B}, \varphi_{\text{expl}})$.
32 Compute $\psi \leftarrow \text{ExtractSend}(\varphi_{\text{expl}}, x)$.
33 **return** ψ as a solution of the Send problem.

Algorithm 22: Algorithm $\mathcal{B}_t$

Input: An instance $x \leftarrow \text{C-INKE}(1^\lambda)$.**Output:** A solution of the Recv problem or the Send problem**Subroutine:** An adversary $\mathcal{A}$ solving C-INKE and the RecoverKey algorithm

```

1 Set a table  $\mathcal{T} \leftarrow \{\}$ .
2 Set a variable  $\sigma_{\text{known}} \leftarrow \perp$ .
3 for  $i \leftarrow 1$  to  $t$  do
4    $z \leftarrow \text{RecoverKey}(x, \mathcal{T}, \sigma_{\text{known}}, \mathcal{A})$ . // Algorithm 21
5   if  $z \neq \perp$  then return  $z$ 
6 return  $\perp$ .
```
